

Module 1 – Active Directory Administration

Task 1 – Active Directory Environment Verification

Objective

Verify that the Active Directory domain, forest, and domain controller are operating correctly before performing any administrative changes.

Background

Before making changes to an Active Directory environment, administrators verify that the domain infrastructure is healthy. This ensures they are connected to the correct domain, confirms the functional levels of the domain and forest, and verifies the domain controller responsible for authentication and directory services.

Systems Used

Component	Value
Server	DC01
Operating System	Windows Server 2022
Domain	benlab.local
Domain Controller	DC01.benlab.local

Commands Used

whoami

Get-ADDomain | Select DNSRoot,DomainMode,PDCEmulator,RIDMaster,InfrastructureMaster

Get-ADForest | Select RootDomain,ForestMode,Domains

Get-ADDomainController | Select HostName,IPv4Address,Site,IsGlobalCatalog

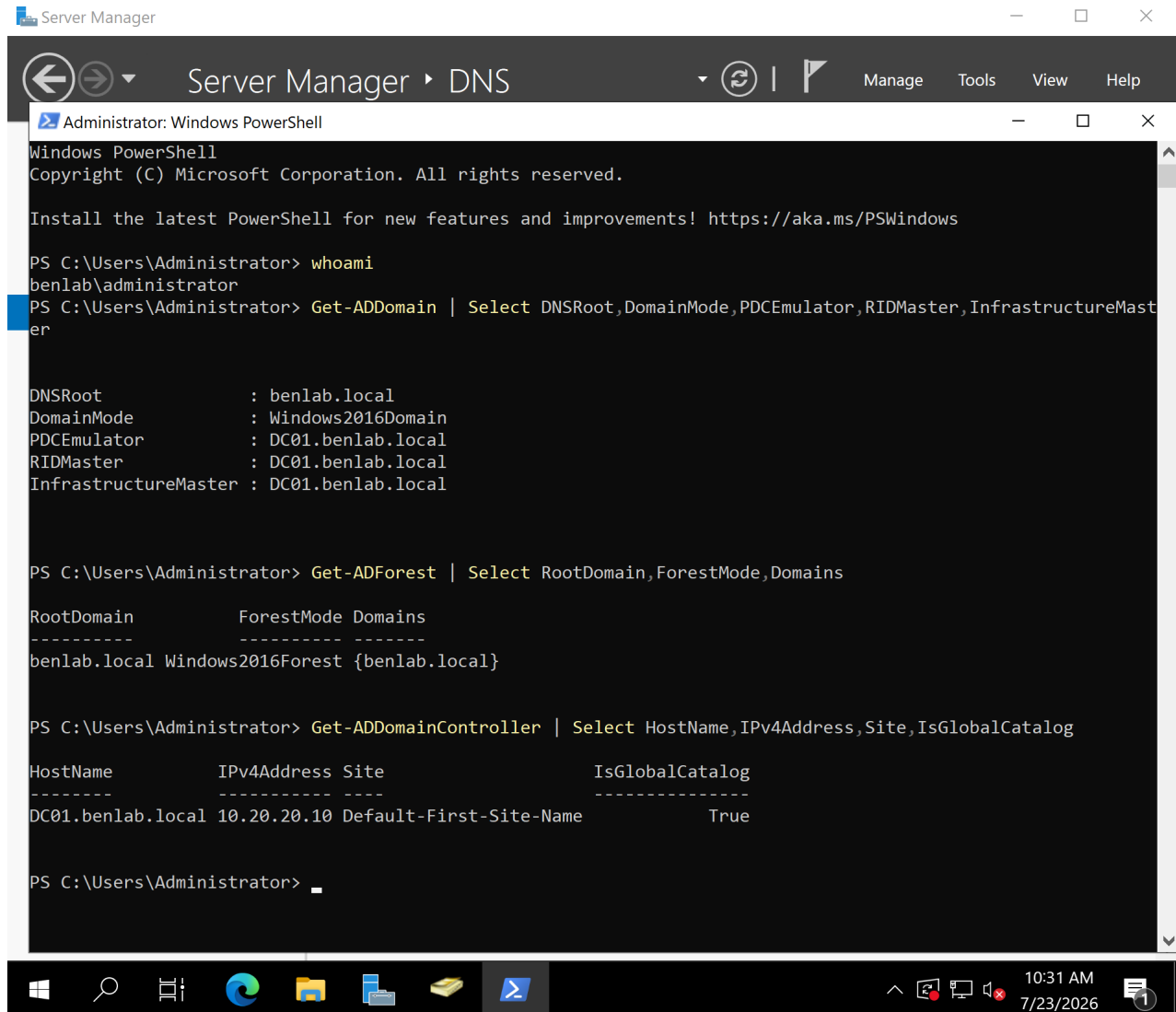
Verification

The verification confirmed:

- The Active Directory domain is **benlab.local**.
- The domain functional level is **Windows Server 2016**.

- DC01 holds the PDC Emulator, RID Master, and Infrastructure Master FSMO roles.
- The forest root domain is **benlab.local**.
- DC01 is functioning as a Global Catalog server.
- The domain controller IP address is **10.20.20.10**.

Screenshot



The screenshot shows the Windows Server Manager interface with the 'DNS' console tree selected. Overlaid on this is a Windows PowerShell window running as Administrator. The PowerShell session displays the output of three commands used to verify the Active Directory environment:

```
PS C:\Users\Administrator> whoami
benlab\Administrator

PS C:\Users\Administrator> Get-ADDomain | Select DNSRoot,DomainMode,PDCEmulator,RIDMaster,InfrastructureMaster

DNSRoot           : benlab.local
DomainMode         : Windows2016Domain
PDCEmulator        : DC01.benlab.local
RIDMaster          : DC01.benlab.local
InfrastructureMaster : DC01.benlab.local

PS C:\Users\Administrator> Get-ADForest | Select RootDomain,ForestMode,Domains

RootDomain      ForestMode Domains
-----
benlab.local    Windows2016Forest {benlab.local}

PS C:\Users\Administrator> Get-ADDomainController | Select HostName,IPv4Address,Site,IsGlobalCatalog

HostName      IPv4Address Site              IsGlobalCatalog
-----
DC01.benlab.local 10.20.20.10 Default-First-Site-Name True

PS C:\Users\Administrator>
```

The taskbar at the bottom shows the system clock as 10:31 AM on 7/23/2026.

M1-T1-Environment-Verification.png

Lessons Learned

Active Directory should always be verified before making administrative changes. Confirming the domain configuration, forest information, FSMO roles, and domain controller health reduces the risk of making configuration changes in the wrong environment.

Task 2 – Enable Active Directory Recycle Bin

Objective

Enable the Active Directory Recycle Bin to provide rapid recovery of deleted Active Directory objects.

Background

The Active Directory Recycle Bin is a Microsoft feature that allows administrators to restore deleted users, groups, computers, and organizational units without performing an authoritative restore from backup. When enabled, deleted objects retain important attributes such as group memberships, security identifiers (SIDs), and permissions.

Unlike older recovery methods, the Recycle Bin significantly reduces recovery time and administrative effort.

Important Note

The Active Directory Recycle Bin is a **one-way feature**.

Once enabled, it cannot be disabled.

Because of this, administrators verify the Active Directory environment before activation.

Steps Performed

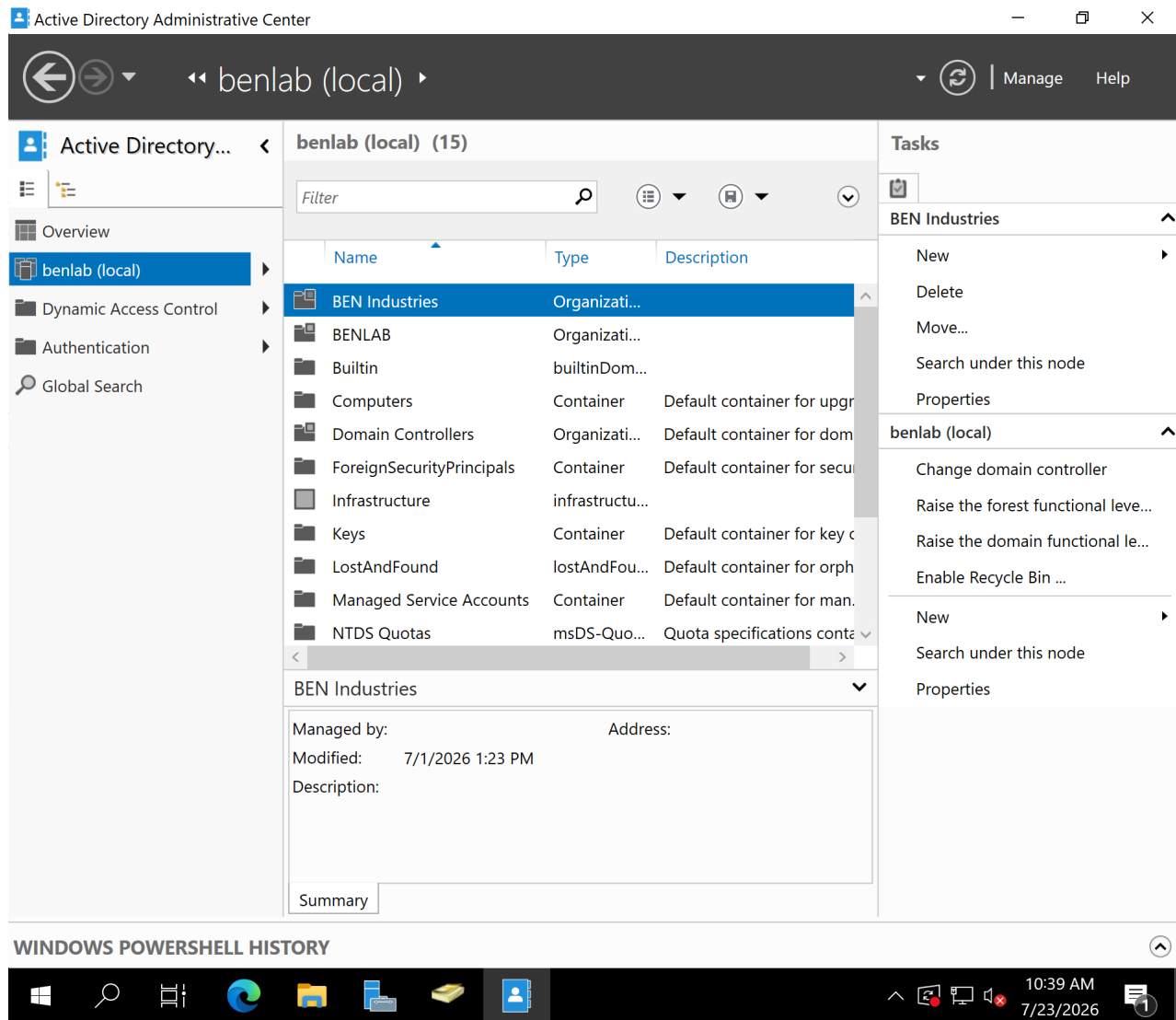
1. Opened **Active Directory Administrative Center**.
 2. Selected the **benlab.local** domain.
 3. Located **Enable Recycle Bin**.
 4. Confirmed the warning dialog.
 5. Enabled the Active Directory Recycle Bin.
 6. Refreshed the Administrative Center.
 7. Verified successful activation.
-

Verification

The following confirms successful activation:

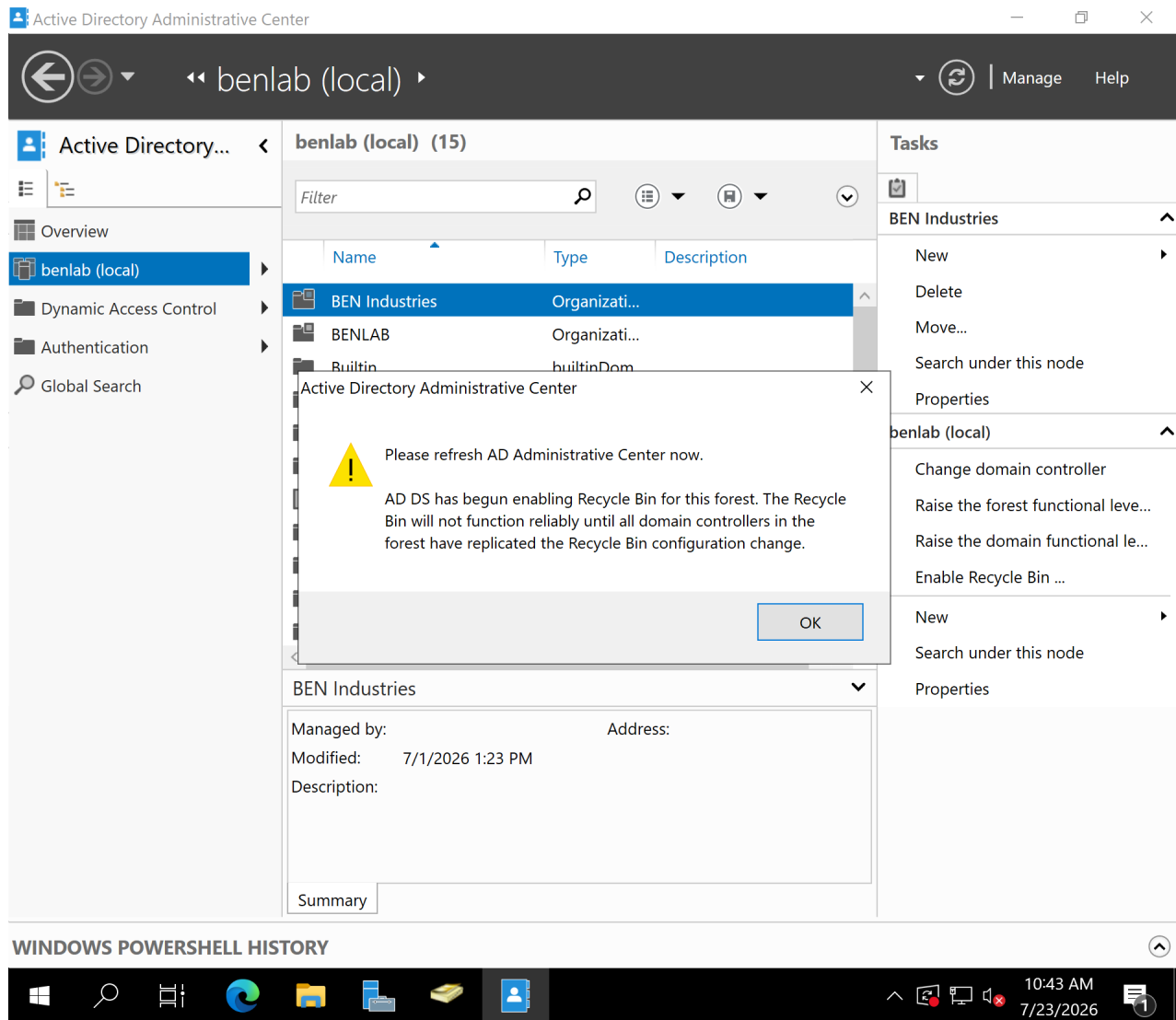
- The **Deleted Objects** container appeared.
- The **Enable Recycle Bin** option became unavailable.
- Active Directory accepted the forest-wide configuration change.

Screenshots



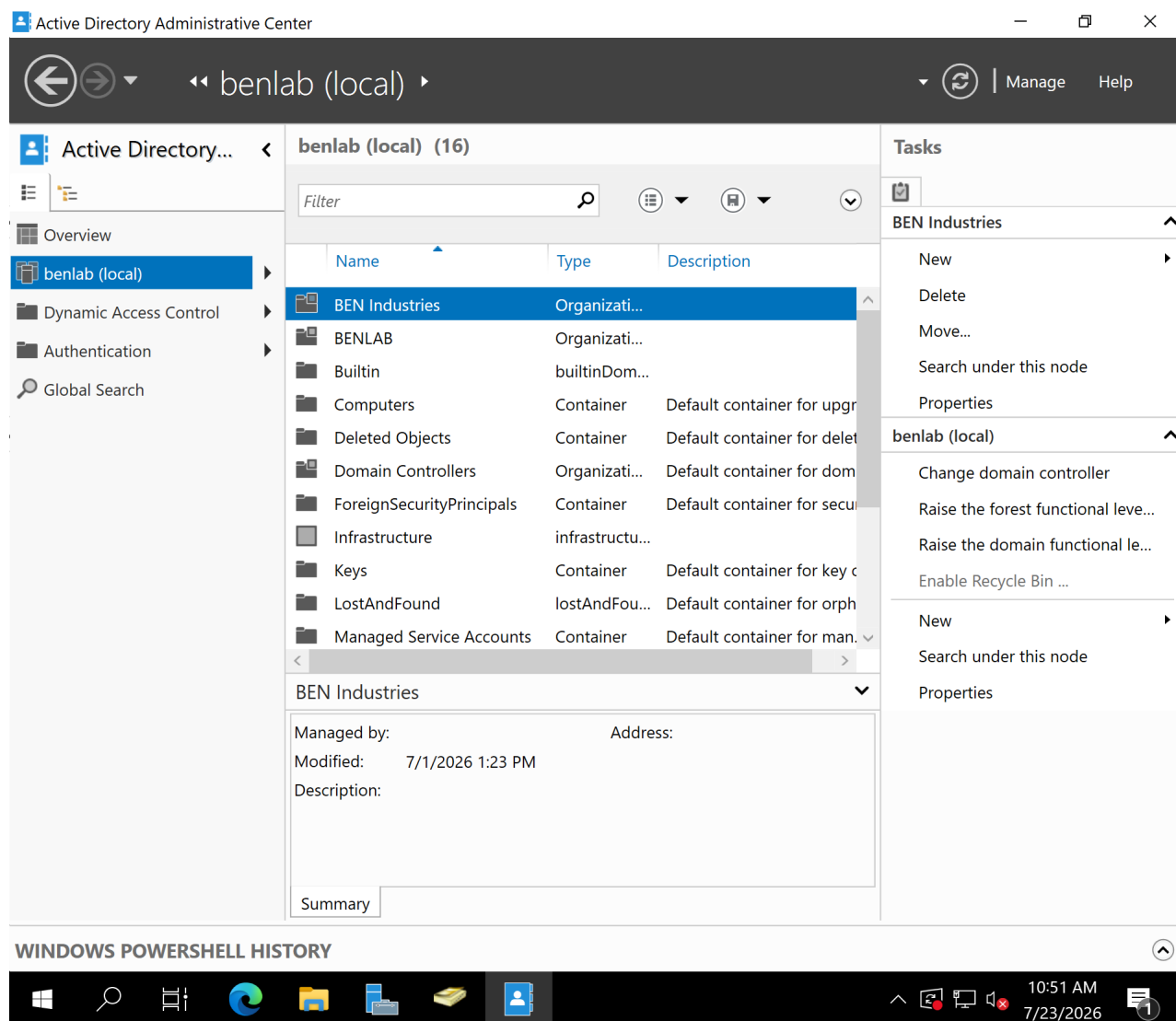
M1-T2-Before-Enable-RecycleBin.png

(Administrative Center showing the Enable Recycle Bin option.)



M1-T2-Enable-RecycleBin-Confirmation.png

(Confirmation dialog indicating the feature is being enabled.)



M1-T2-RecycleBin-Enabled.png

(Administrative Center showing the Deleted Objects container after activation.)

Lessons Learned

Enabling the Active Directory Recycle Bin is considered a best practice because it provides a fast and reliable method for recovering accidentally deleted Active Directory objects while preserving their attributes and security information. Understanding that this is a permanent forest-wide configuration reinforces the importance of validating the environment before making infrastructure changes.

Task 3 – Create a Fine-Grained Password Policy (FGPP)

Objective

Create a **Fine-Grained Password Policy (FGPP)** using a **Password Settings Object (PSO)** to enforce stronger password and account lockout requirements for members of the **GG_IT** security group.

Background

The Default Domain Password Policy applies the same password and account lockout requirements to every user in an Active Directory domain. However, enterprise environments often require different security standards for different types of accounts.

Fine-Grained Password Policies (FGPP) allow administrators to create multiple password policies within the same domain and apply them to specific users or security groups. This provides greater flexibility and improves security by allowing privileged accounts to follow stricter password requirements than standard user accounts.

In this task, a Password Settings Object (PSO) was created to strengthen password security for members of the **GG_IT** security group.

Systems Used

Component	Value
Server	DC01
Operating System	Windows Server 2022
Domain	benlab.local
Security Group	GG_IT
Policy Type	Password Settings Object (PSO)

Configuration Summary

Setting	Value
Policy Name	PSO-IT-Administrators
Precedence	1
Minimum Password Length	15 characters
Password History	24 remembered passwords
Password Complexity	Enabled
Reversible Encryption	Disabled
Minimum Password Age	1 day

Setting	Value
Maximum Password Age	30 days
Account Lockout	Enabled
Failed Logon Attempts	3
Reset Failed Attempts	30 minutes
Lockout Duration	30 minutes
Applied To	GG_IT

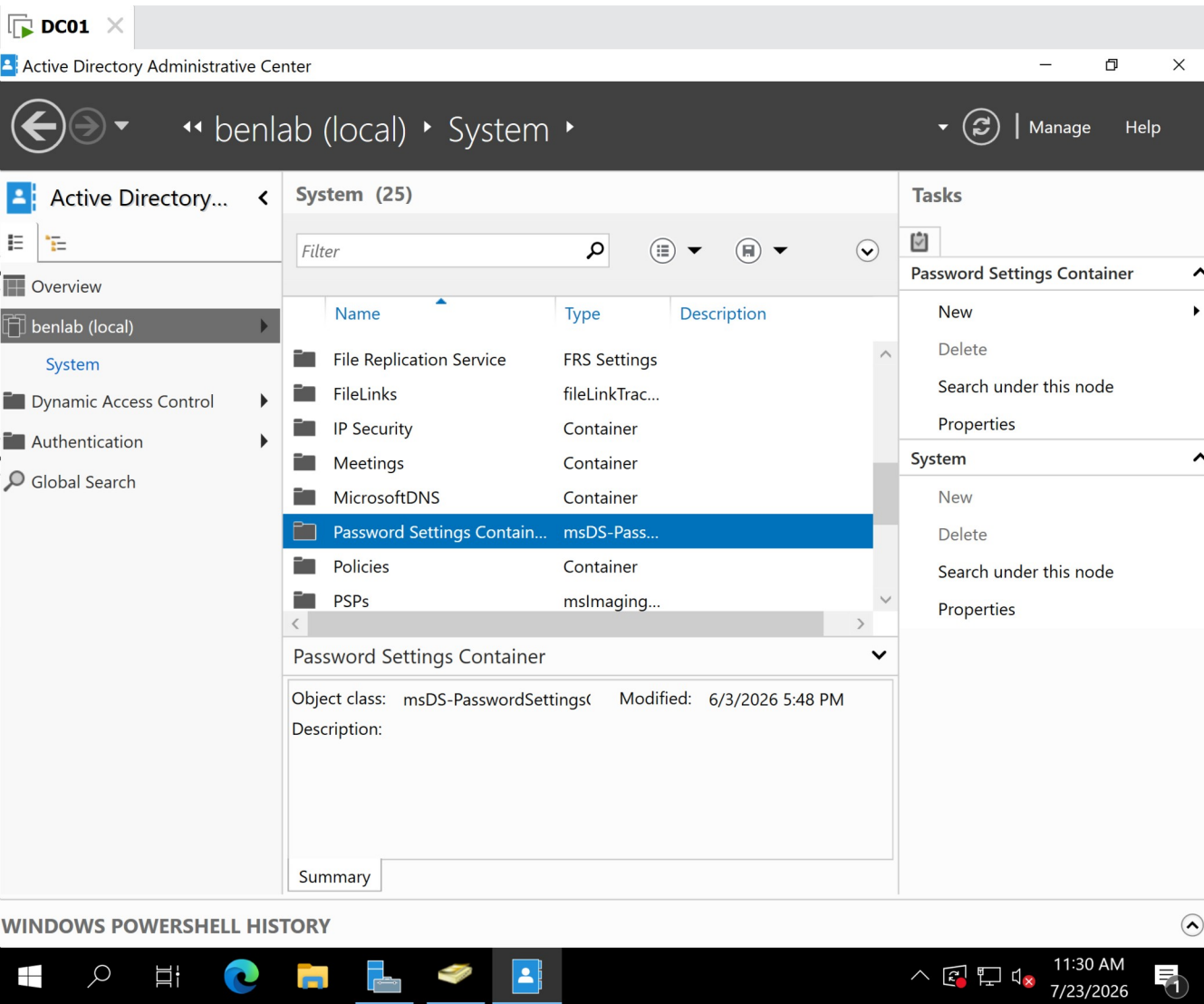
Steps Performed

1. Verified that the **GG_IT** security group already existed in Active Directory.
 2. Opened the **Password Settings Container** using the Active Directory Administrative Center.
 3. Created a new Password Settings Object named **PSO-IT-Administrators**.
 4. Configured stronger password complexity, password history, and password age requirements.
 5. Configured an account lockout policy to protect administrative accounts from brute-force attacks.
 6. Added the **GG_IT** security group under **Directly Applies To**.
 7. Saved the Password Settings Object.
 8. Verified that the policy was successfully created and stored in the Password Settings Container.
-

Verification

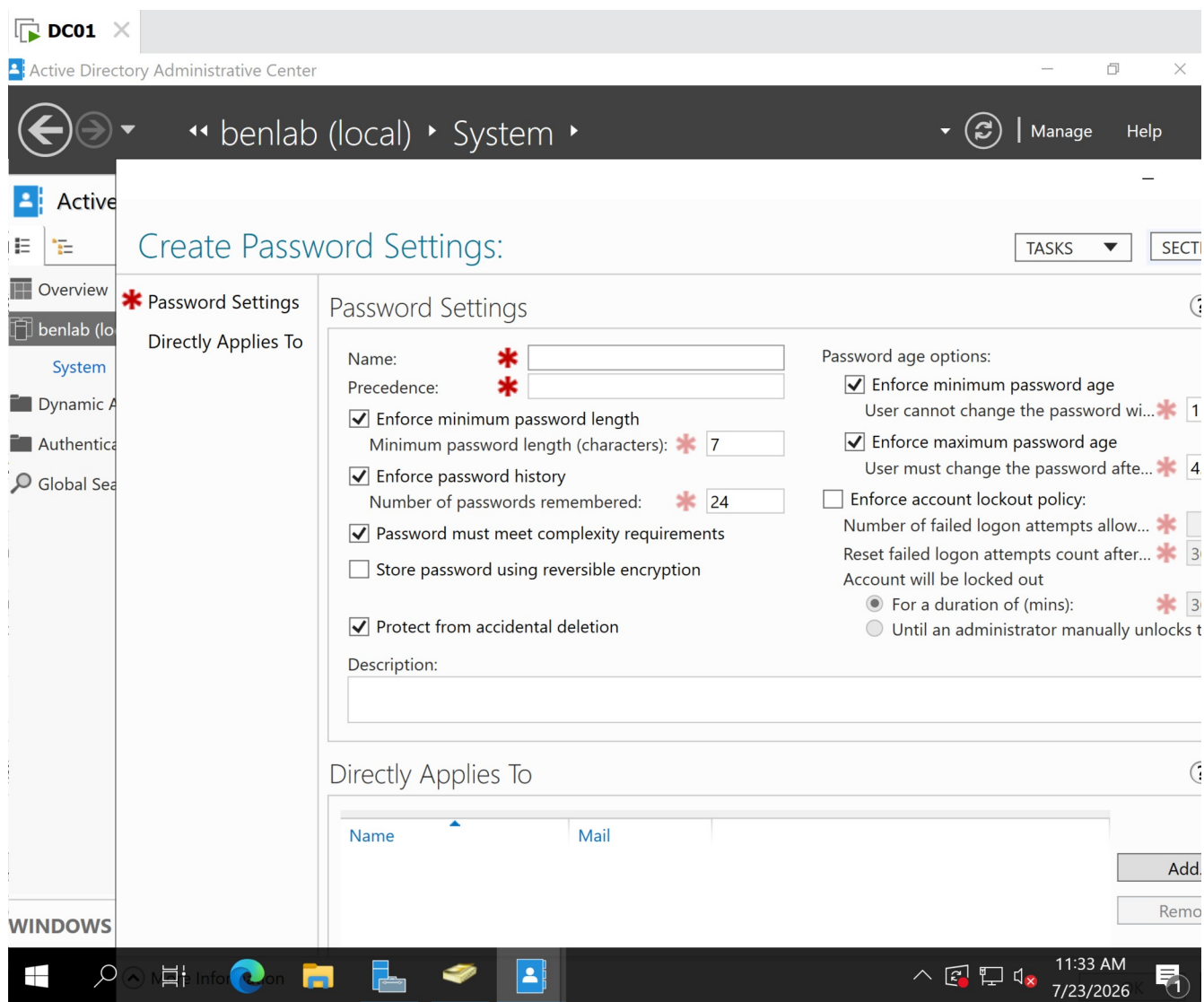
Verification confirmed that the Password Settings Object **PSO-IT-Administrators** was successfully created with a precedence value of **1** and was configured with stronger password and account lockout settings. The policy was linked to the **GG_IT** security group, ensuring that all current and future members of the group will automatically receive the enhanced password policy.

Screenshots



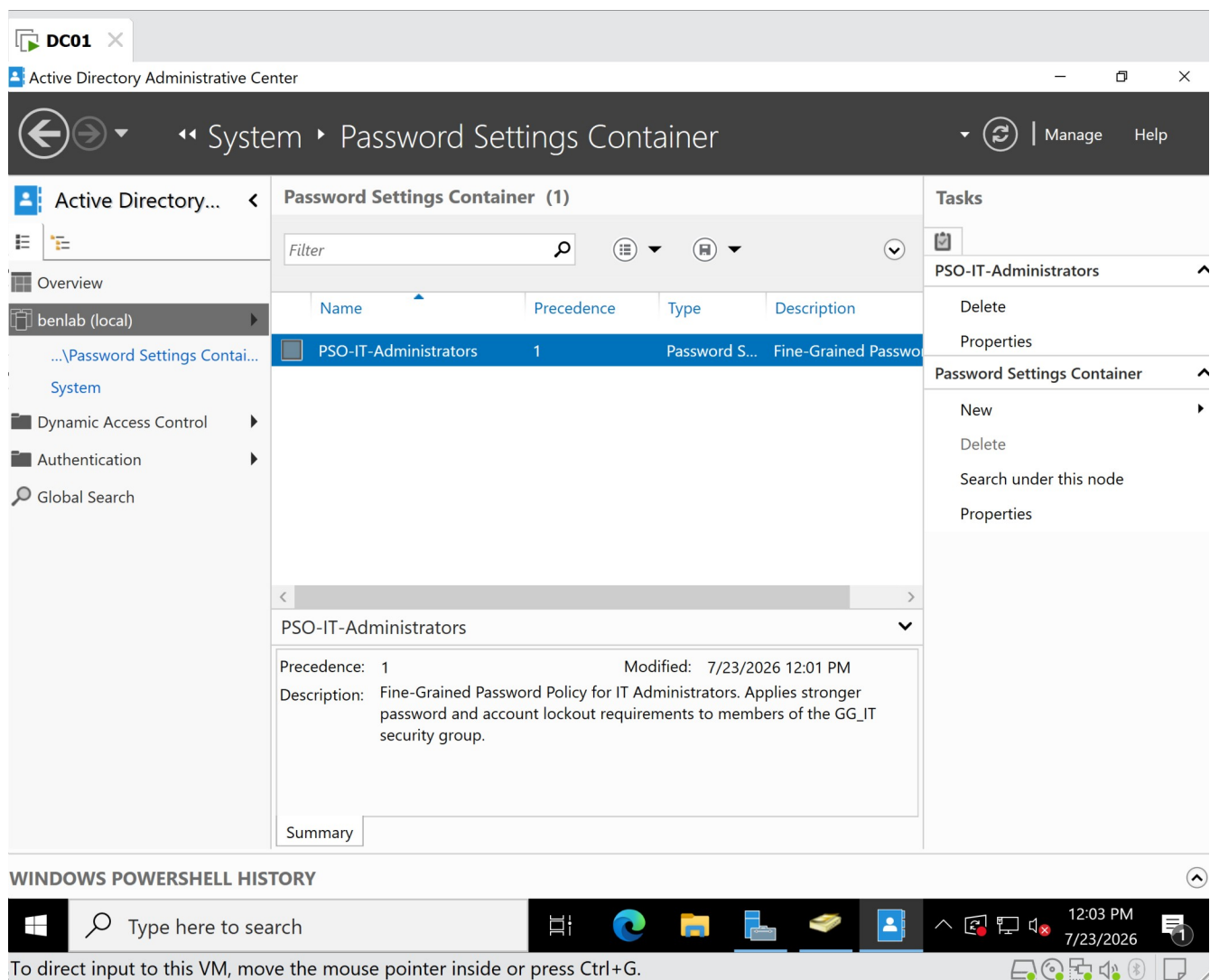
M1-T3-GG_IT-Verification.png

Verified that the GG_IT security group exists before creating the Password Settings Object.



M1-T3-Password-Settings-Container.png

Verified the location of the Password Settings Container prior to creating the Fine-Grained Password Policy.



M1-T3-PSO-Created.png

Verified that the Password Settings Object was successfully created and stored in the Password Settings Container.

Key Concepts Learned

- A **Password Settings Object (PSO)** stores a Fine-Grained Password Policy in Active Directory.
- Fine-Grained Password Policies allow different password requirements for different users or security groups without modifying the Default Domain Policy.
- **Precedence** determines which Password Settings Object is applied when multiple policies affect the same user. The **lowest numerical value has the highest priority**.
- Applying a Password Settings Object to a **security group** is more scalable than assigning it to individual users because all current and future members automatically inherit the policy.

- Strong password policies and account lockout settings reduce the risk of brute-force attacks against privileged accounts.
-

Lessons Learned

This task demonstrated how Fine-Grained Password Policies provide flexibility and improve security in enterprise Active Directory environments. Rather than applying identical password requirements to every user, administrators can enforce stronger authentication standards for privileged accounts while maintaining separate policies for standard users. Applying the policy to a security group also simplifies ongoing administration and supports future growth without requiring additional configuration.

Task 4 – Delegate Control of an Organizational Unit (OU)

Objective

Delegate administrative control of the **Finance Organizational Unit (OU)** to a designated Help Desk technician without granting Domain Administrator privileges. This delegation enables the technician to perform common user account management tasks while maintaining the Principle of Least Privilege.

Background

In enterprise Active Directory environments, it is not considered a security best practice to grant Domain Administrator privileges to Help Desk personnel. Instead, administrators delegate only the permissions required for specific job responsibilities.

The **Delegation of Control Wizard** simplifies assigning administrative permissions to users or groups for a specific Organizational Unit (OU). This approach limits administrative access to only the required resources and reduces the risk of accidental or unauthorized changes elsewhere in the domain.

In this task, administrative control of the **Finance OU** was delegated to **Daniel Scott**, allowing him to manage Finance user accounts without becoming a Domain Administrator.

Systems Used

Component	Value
Server	DC01

Component	Value
Operating System	Windows Server 2022
Domain	benlab.local
Organizational Unit	Finance
Delegated User	Daniel Scott
Administrative Tool	Active Directory Users and Computers (ADUC)

Delegation Scenario

BEN Industries promoted **Daniel Scott** to a Help Desk Technician role. As part of his responsibilities, he must support users within the **Finance** department.

To follow Microsoft's Principle of Least Privilege, Daniel Scott was granted administrative permissions only for the Finance OU instead of being added to the **Domain Admins** group.

This delegation allows Daniel Scott to perform common Help Desk tasks while limiting administrative access to the Finance department.

Delegated Permissions

The following permissions were assigned using the Delegation of Control Wizard:

Permission	Purpose
Create, delete, and manage user accounts	Allows management of Finance department user accounts.
Reset user passwords and force password change at next logon	Enables password recovery and enforces secure password changes.
No additional administrative permissions were granted.	

Steps Performed

1. Opened **Active Directory Users and Computers (ADUC)**.
2. Navigated to **BEN Industries** → **Departments** → **Finance**.
3. Right-clicked the **Finance** Organizational Unit.
4. Launched the **Delegation of Control Wizard**.
5. Added **Daniel Scott** as the delegated administrator.
6. Selected the following delegated tasks:

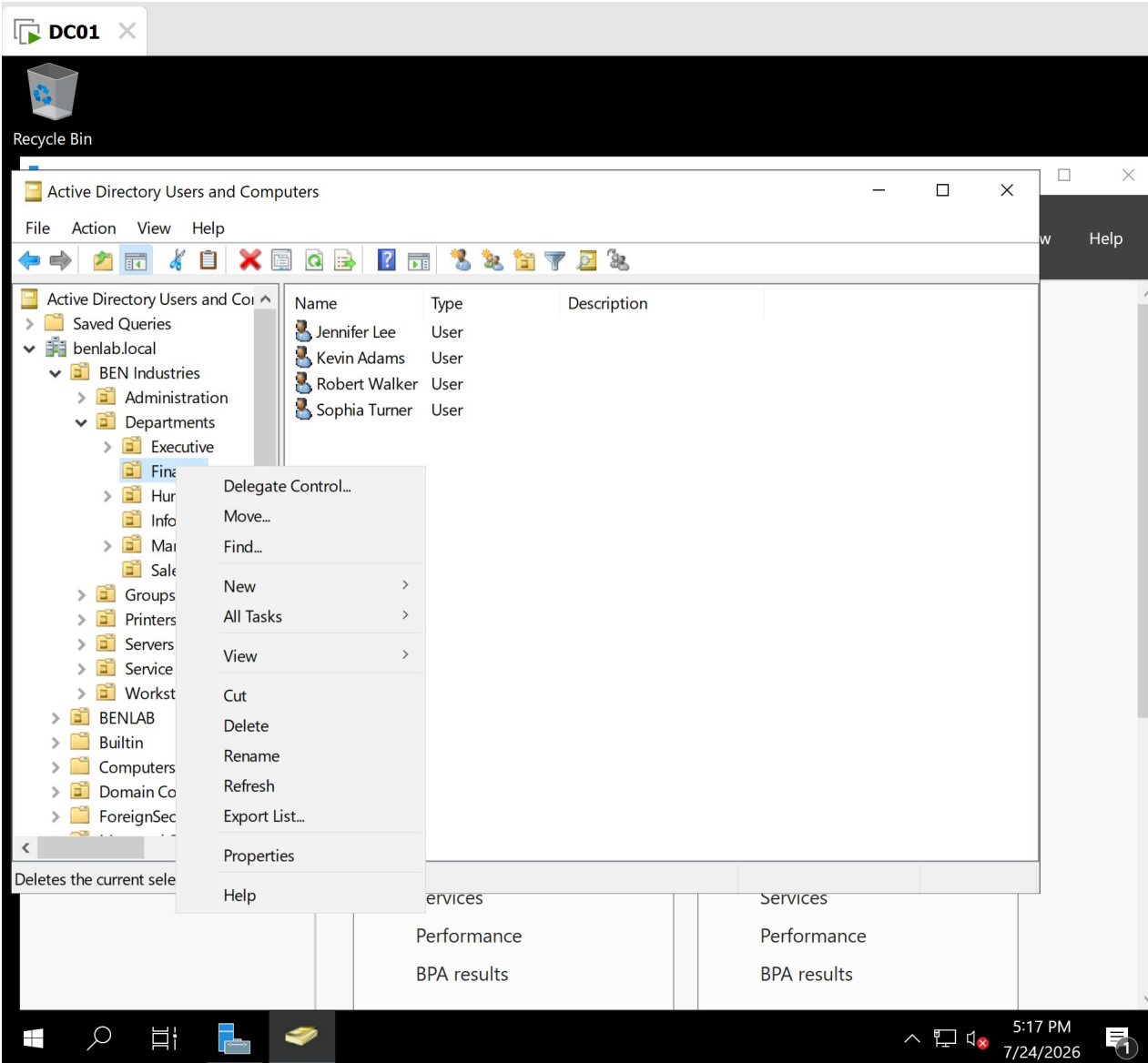
- Create, delete, and manage user accounts.
 - Reset user passwords and force password change at next logon.
7. Completed the Delegation of Control Wizard.
 8. Opened the **Finance OU Properties**.
 9. Verified that **Daniel Scott** appeared on the **Security** tab, confirming that the delegated permissions had been successfully applied.
-

Verification

Verification confirmed that the delegation was successfully applied to the Finance Organizational Unit.

The Security properties of the Finance OU displayed **Daniel Scott** as a delegated security principal. This confirms that Daniel Scott now has the assigned administrative permissions for the Finance OU without receiving Domain Administrator privileges.

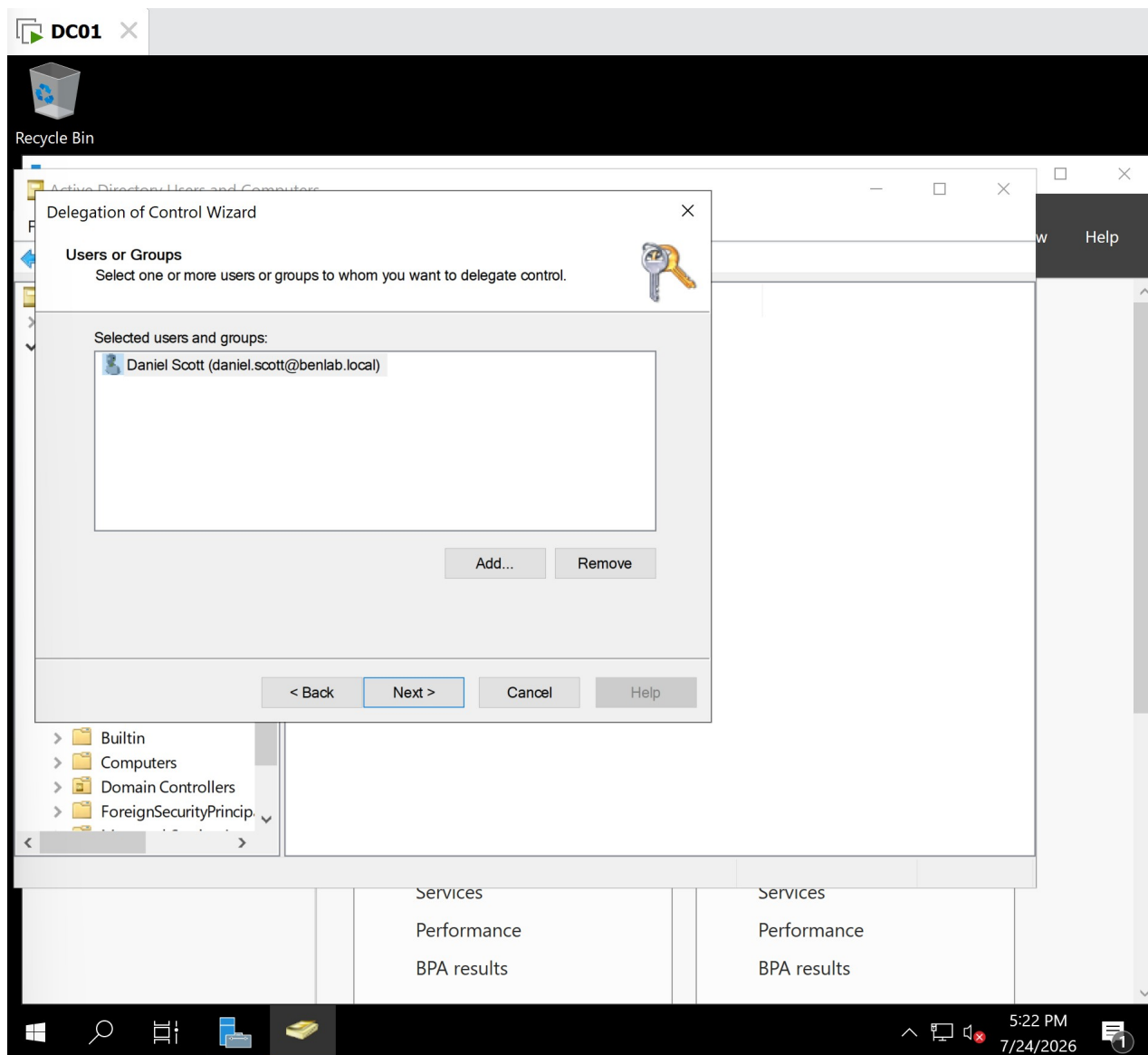
Screenshots



M1-T4-Delegate-Control-Start.png

Description

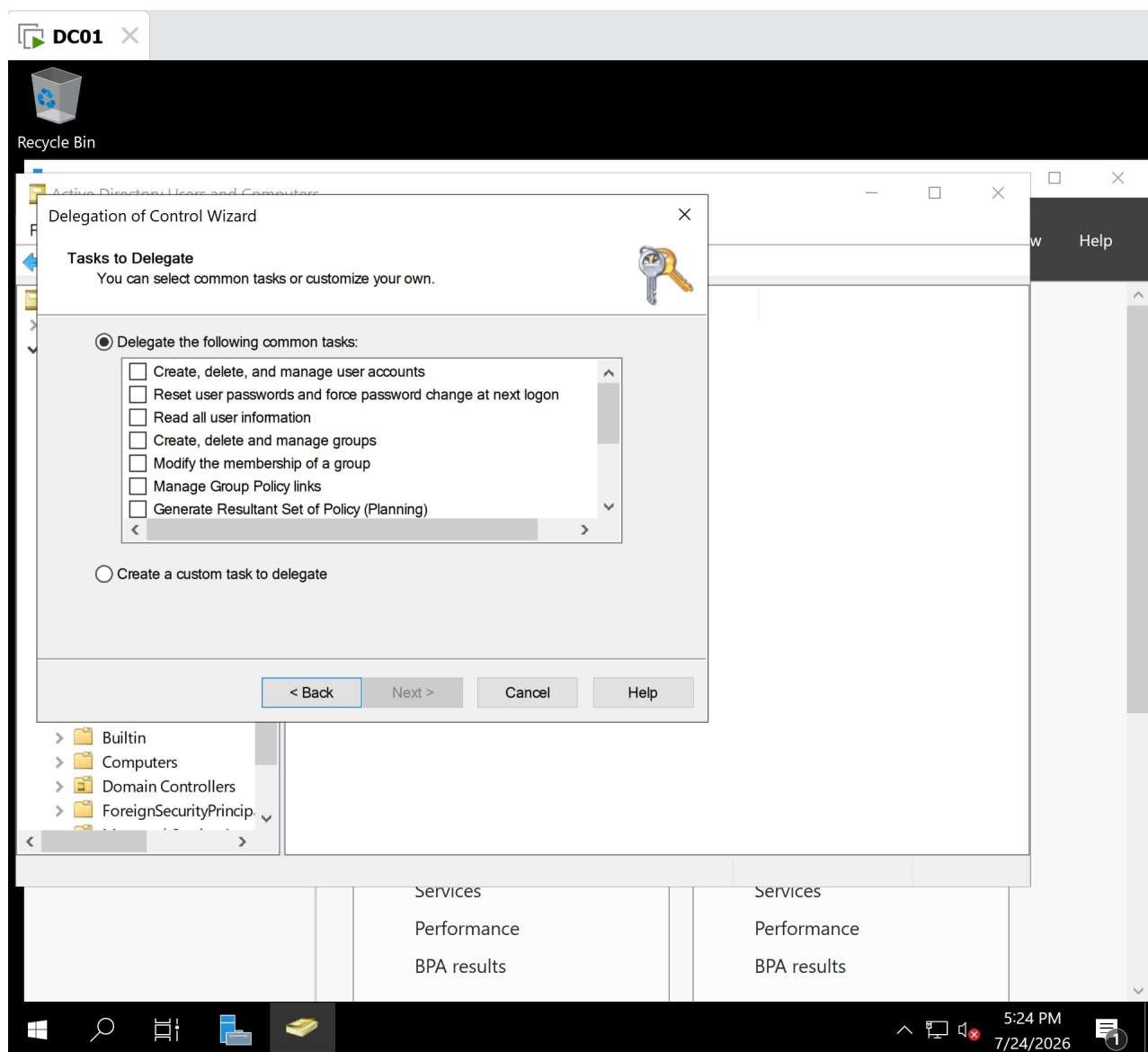
Opened the Finance Organizational Unit and launched the Delegation of Control Wizard.



M1-T4-Delegate-User-Selected.png

Description

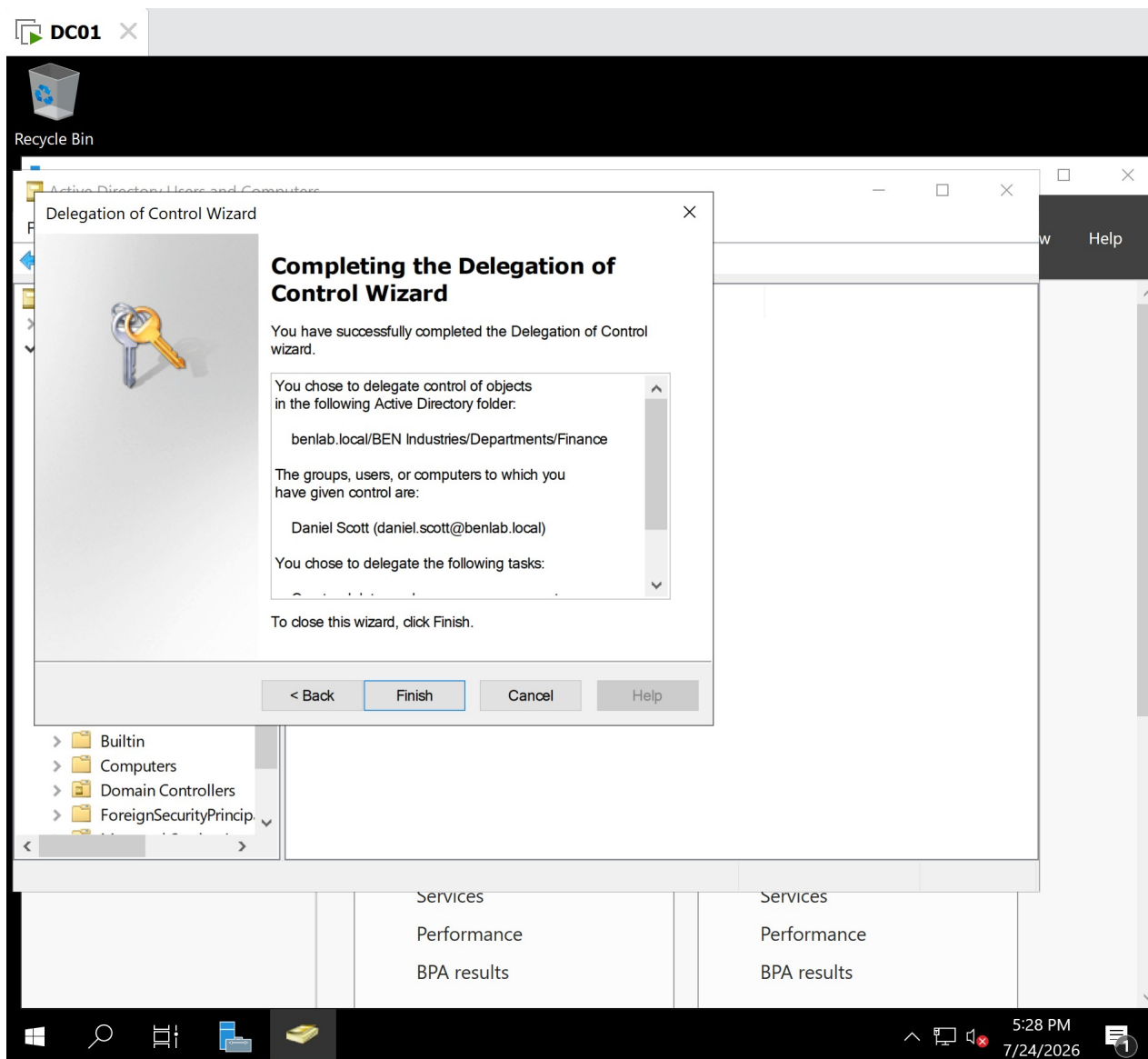
Selected Daniel Scott as the delegated administrator for the Finance Organizational Unit.



M1-T4-Delegation-Tasks.png

Description

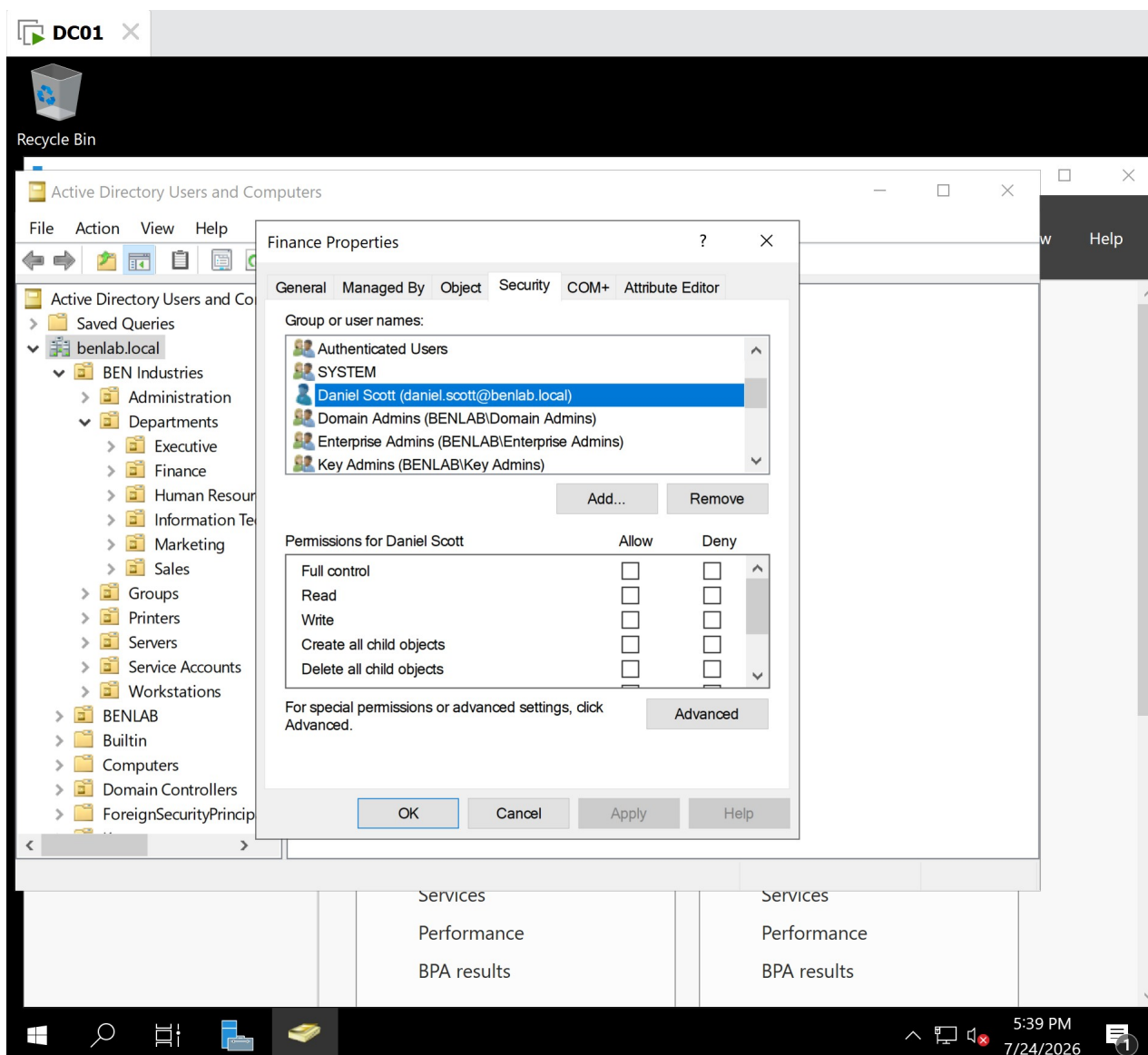
Selected the delegated administrative tasks for Daniel Scott using the Delegation of Control Wizard.



M1-T4-Delegation-Summary.png

Description

Reviewed the delegation summary before applying the configuration.



M1-T4-Delegation-Verification.png

Description

Verified that Daniel Scott appears in the Security permissions of the Finance Organizational Unit after the delegation was successfully applied.

Key Concepts Learned

- Delegation of Control allows administrators to assign specific administrative permissions without granting Domain Administrator rights.
- The **Delegation of Control Wizard** provides a secure method for assigning common administrative tasks.

- Organizational Units (OUs) enable administrators to delegate permissions at a departmental level.
 - Following the **Principle of Least Privilege** reduces security risks by granting users only the permissions necessary to perform their assigned duties.
 - Verifying delegated permissions after configuration ensures that the intended security settings have been successfully applied.
-

Lessons Learned

This task demonstrated how Active Directory delegation supports secure administrative practices in enterprise environments. Rather than assigning excessive privileges, administrators can delegate only the permissions required for specific responsibilities. By granting Daniel Scott limited administrative control over the Finance Organizational Unit, the environment remains secure while allowing Help Desk personnel to efficiently support end users. This approach improves security, accountability, and operational efficiency within the organization.

Administrator's Note

Enterprise Best Practice: Administrative permissions should be delegated to security groups whenever possible instead of individual user accounts. In this lab, Daniel Scott was delegated permissions directly to simplify the learning exercise. In a production environment, Microsoft recommends delegating permissions to a Help Desk security group (for example, **GG_HelpDesk**) so that administrators can manage access by adding or removing group members rather than modifying permissions on the OU itself.

Task 5 – Active Directory Administrative Center (ADAC) Administration

Objective

Explore and use the **Active Directory Administrative Center (ADAC)** to manage Active Directory objects, navigate Organizational Units (OUs), view user properties, and perform Global Search. The objective of this task is to become familiar with Microsoft's modern Active Directory management interface and understand its advantages over the traditional Active Directory Users and Computers (ADUC) console.

Background

The **Active Directory Administrative Center (ADAC)** is Microsoft's modern management interface for Active Directory Domain Services (AD DS). While **Active Directory Users and Computers (ADUC)** remains widely used for routine administration, ADAC introduces additional features that simplify management and improve administrator productivity.

ADAC integrates advanced capabilities such as **Global Search**, **Fine-Grained Password Policy management**, **Active Directory Recycle Bin**, and **Windows PowerShell History**. These features allow administrators to perform administrative tasks more efficiently while also learning the PowerShell commands associated with many graphical operations.

Systems Used

Component	Value
Server	DC01
Operating System	Windows Server 2022
Domain	benlab.local
Management Tool	Active Directory Administrative Center (ADAC)
Test User	Jennifer Lee

Features Explored

During this task, the following ADAC features were explored:

- Active Directory Overview
- Organizational Unit navigation
- Tasks pane
- Summary pane
- User Properties
- Global Search
- Windows PowerShell History

- Password Settings
-

Steps Performed

1. Opened **Active Directory Administrative Center (ADAC)** from **Server Manager** → **Tools**.
 2. Reviewed the ADAC Overview page and its built-in administrative tools.
 3. Navigated to **BEN Industries** → **Departments**.
 4. Selected the **Finance** Organizational Unit.
 5. Reviewed the Tasks pane and Summary pane.
 6. Opened the **Jennifer Lee** user account.
 7. Examined the user property sections, including:
 - Account
 - Member Of
 - Password Settings
 8. Used **Global Search** to locate Jennifer Lee.
 9. Reviewed the available administrative actions and Windows PowerShell History panel.
-

Verification

Verification confirmed that Active Directory Administrative Center was functioning correctly.

The Global Search feature successfully located the Jennifer Lee user account within the domain. User properties displayed correctly, and administrative tasks such as password reset, group membership management, and password policy viewing were available directly from the Tasks pane. This demonstrated ADAC's ability to provide centralized and efficient Active Directory administration.

ADUC vs ADAC Comparison

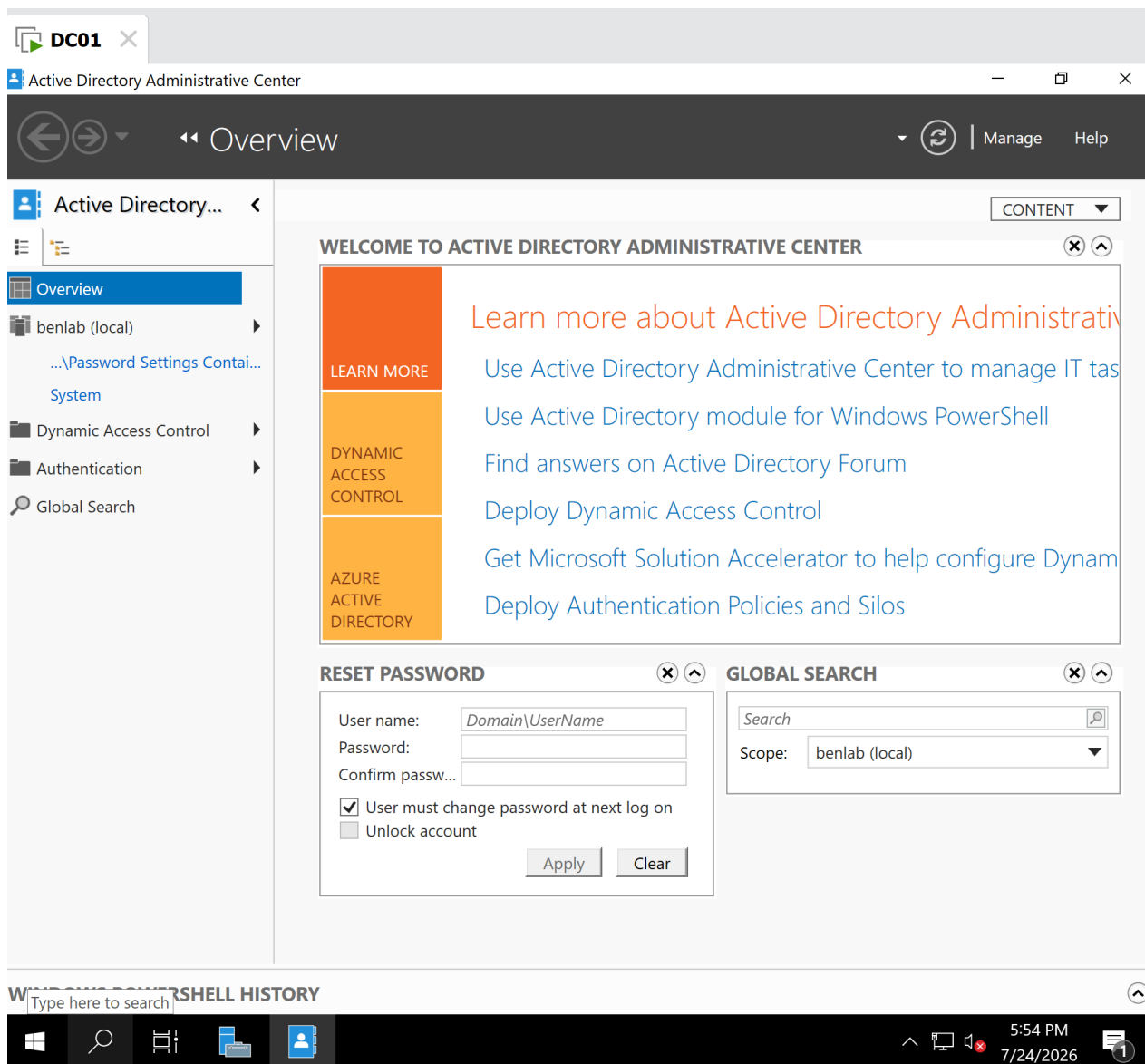
Active Directory Users and Computers (ADUC)

Traditional MMC console
Fast and lightweight
Right-click context menus
Basic object management
Limited search capabilities
No integrated PowerShell history
Limited Fine-Grained Password Policy management
No Recycle Bin management

Active Directory Administrative Center (ADAC)

Modern Microsoft management interface
Feature-rich and modern
Integrated Tasks pane
Advanced object management
Domain-wide Global Search
Built-in Windows PowerShell History
Full Fine-Grained Password Policy support
Built-in Active Directory Recycle Bin support

Screenshots



M1-T5-ADAC-Overview.png

Description

Opened the Active Directory Administrative Center and reviewed the Overview dashboard.

DC01 x

Active Directory Administrative Center

← BEN Industries ▸ Departments ▸ Manage Help

Active Directory... <

Departments (6)

Filter

Name	Type	Description
Executive	Organizati...	
Finance	Organizati...	
Human Resources	Organizati...	
Information Technology	Organizati...	
Marketing	Organizati...	
Sales	Organizati...	

Finance

Managed by: Address:

Modified: 7/24/2026 5:31 PM

Description:

Summary

Tasks

Finance

- New
- Delete
- Move...
- Search under this node
- Properties

Departments

- New
- Delete
- Move...
- Search under this node
- Properties

WINDOWS POWERSHELL HISTORY

Type here to search

6:09 PM 7/24/2026

M1-T5-ADAC-Finance-OU.png

Description

Navigated to the Finance Organizational Unit and examined the Tasks pane and Summary pane.

DC01 x

Active Directory Administrative Center

Jennifer Lee

TASKS ▼ SECTIONS ▼

Account

Organization

Member Of

Password Settings

Profile

Policy

Silo

Extensions

Account

First name: Jennifer

Middle initials:

Last name: Lee

Full name: * Jennifer Lee

User UPN login: jlee @ benlab.local

User SamAccountN... benlab * jlee

☐ Protect from accidental deletion

Account expires: ☒ Never ☐ End of

Password options:

☐ User must change password at next log on

☐ Other password options

☐ Microsoft Passport or smart card is required for in...

☐ Password never expires

☐ User cannot change password

Encryption options: ▼

Other options: ▼

Log on hours... Log on to...

Organization

Display name: Jennifer Lee

Office:

E-mail:

Web page:

Other web pages...

Phone numbers:

Job title: Senior Accountant

Department: Finance

Company: BEN Industries

Manager: Robert Walker Edit... Clear

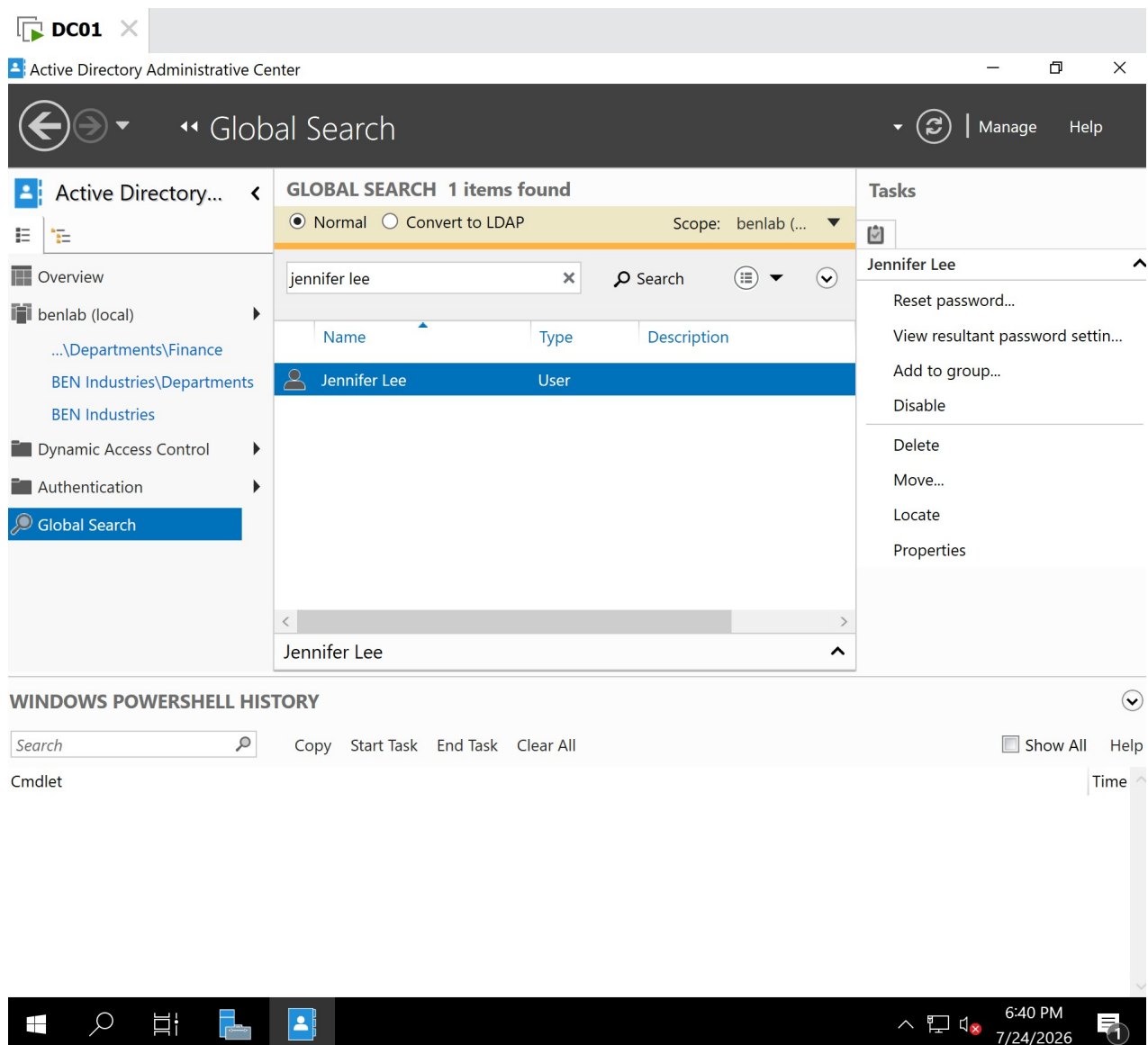
Direct reports: Add...

More Information

OK Cancel

M1-T5-ADAC-User-Properties.png

Description Reviewed Jennifer Lee's user properties, including Account, Member Of, and Password Settings sections.



M1-T5-ADAC-Global-Search.png

Description

Used the Global Search feature to locate Jennifer Lee and reviewed the available administrative actions.

Key Concepts Learned

- Active Directory Administrative Center (ADAC) is Microsoft's modern interface for managing Active Directory.

- ADAC complements rather than replaces Active Directory Users and Computers (ADUC); experienced administrators often use both depending on the task.
 - Global Search allows administrators to locate Active Directory objects across the domain without manually browsing Organizational Units.
 - The **Tasks** pane provides quick access to common administrative actions, improving workflow efficiency.
 - The **Password Settings** section simplifies viewing the effective password policy for a user, including Fine-Grained Password Policies.
 - The **Windows PowerShell History** pane helps administrators learn and automate Active Directory management by exposing PowerShell commands generated through the GUI.
-

Lessons Learned

This task demonstrated the advantages of using Active Directory Administrative Center for modern Active Directory administration. Compared to the traditional ADUC console, ADAC offers a more organized interface, integrated search capabilities, advanced password policy management, and PowerShell integration. Understanding both management tools enables administrators to choose the most appropriate interface for each administrative task and prepares them for managing enterprise Windows Server environments efficiently.

Administrator's Notes

Enterprise Best Practice: Most Windows administrators use both **ADUC** and **ADAC** in their daily work. ADUC is typically preferred for quick, routine administrative tasks, while ADAC is used for advanced management features such as Fine-Grained Password Policies, Active Directory Recycle Bin operations, Global Search, and PowerShell-integrated administration. Becoming proficient with both tools provides greater flexibility and aligns with common enterprise administration practices.

Task 6 – Active Directory PowerShell Administration

Objective

Learn how to use Windows PowerShell to administer Active Directory by importing the Active Directory module, retrieving domain information, querying users and Organizational Units (OUs), viewing user properties, and inspecting group memberships. The objective of this task is to understand how PowerShell complements graphical administration tools by enabling faster, repeatable, and automated management of Active Directory.

Background

Windows PowerShell is Microsoft's command-line automation framework used extensively for Windows Server administration. The **Active Directory PowerShell module** provides administrators with cmdlets to manage users, groups, computers, Organizational Units, Group Policy, and other Active Directory objects.

Unlike graphical tools such as **Active Directory Users and Computers (ADUC)** and **Active Directory Administrative Center (ADAC)**, PowerShell enables administrators to automate repetitive tasks, generate reports, and manage hundreds or thousands of objects efficiently.

Systems Used

Component	Value
Server	DC01
Operating System	Windows Server 2022
Domain	benlab.local
PowerShell Module	ActiveDirectory
Test User	Jennifer Lee (jlee)

PowerShell Cmdlets Used

```
Get-Module ActiveDirectory -ListAvailable
```

Verified that the Active Directory module was installed.

```
Import-Module ActiveDirectory
```

Imported the Active Directory module into the current PowerShell session.

```
Get-ADDomain | Select DNSRoot,NetBIOSName,DomainMode
```

Verified domain configuration.

```
Get-ADUser -Filter * | Select Name
```

Retrieved all Active Directory users.

```
Get-ADUser -Filter * -SearchBase "OU=Finance,OU=Departments,OU=BEN  
Industries,DC=benlab,DC=local" | Select Name
```

Retrieved users located only within the Finance Organizational Unit.

```
Get-ADUser -Filter 'Name -eq "Jennifer Lee"' | Select Name,SamAccountName
```

Identified Jennifer Lee's **SamAccountName**.

```
Get-ADUser jlee -Properties Department,Title,Enabled | Select  
Name,SamAccountName,Department,Title,Enabled
```

Retrieved additional user properties.

```
Get-ADPrincipalGroupMembership jlee | Select Name
```

Displayed Jennifer Lee's Active Directory group memberships.

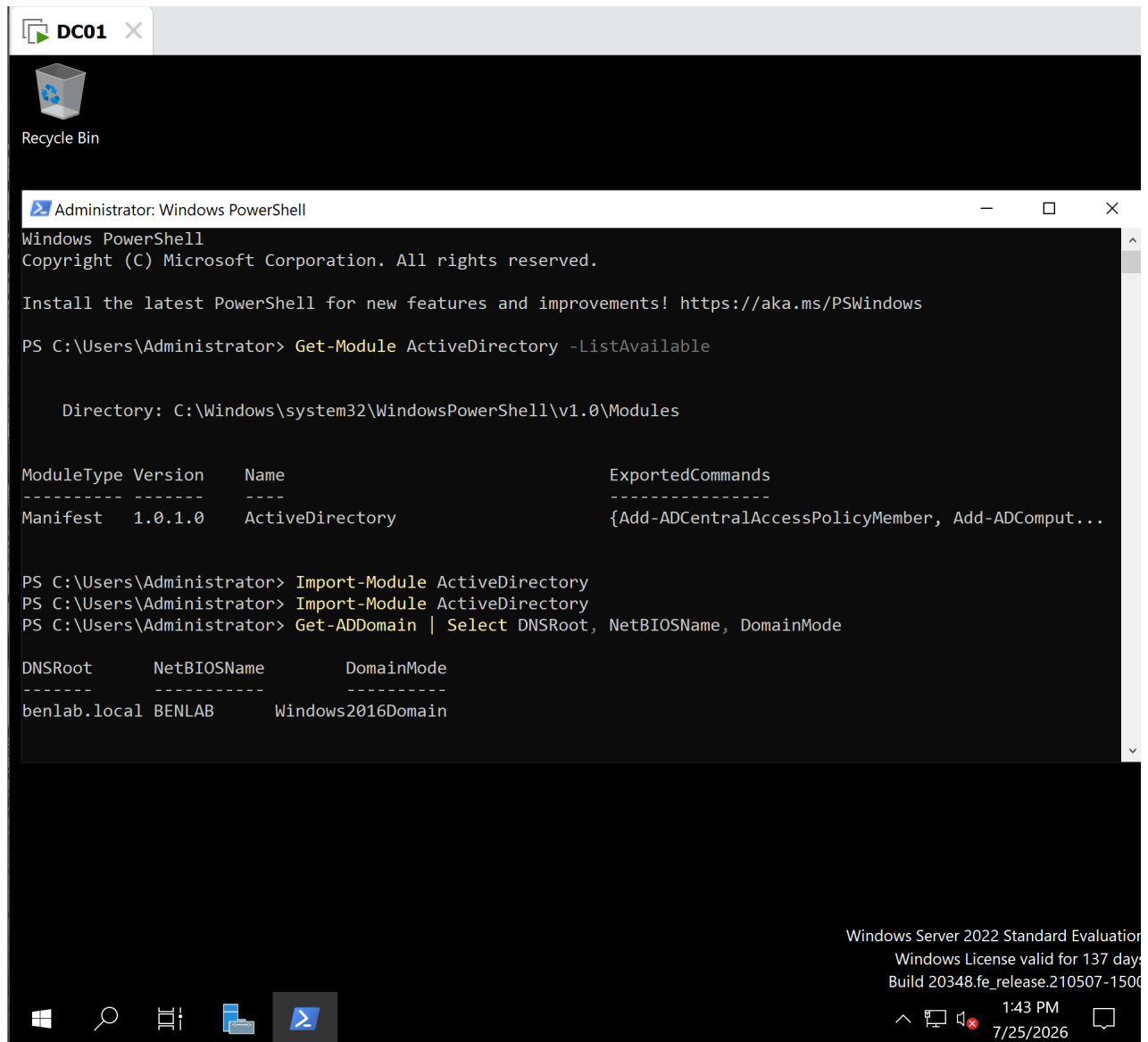
Verification

PowerShell successfully communicated with Active Directory.

The following information was verified:

- Active Directory module installed
 - Module imported successfully
 - Domain configuration retrieved
 - Active Directory users listed
 - Finance OU queried successfully
 - Jennifer Lee's account identified using **SamAccountName**
 - Department and job title displayed
 - Account status verified as enabled
 - Group membership successfully retrieved
-

Screenshots



```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Administrator> Get-Module ActiveDirectory -ListAvailable

Directory: C:\Windows\system32\WindowsPowerShell\v1.0\Modules


ModuleType Version     Name                                ExportedCommands
-----
Manifest    1.0.1.0     ActiveDirectory                    {Add-ADCentralAccessPolicyMember, Add-ADComput...

PS C:\Users\Administrator> Import-Module ActiveDirectory
PS C:\Users\Administrator> Import-Module ActiveDirectory
PS C:\Users\Administrator> Get-ADDomain | Select DNSRoot, NetBIOSName, DomainMode

DNSRoot      NetBIOSName  DomainMode
-----
benlab.local BENLAB      Windows2016Domain

Windows Server 2022 Standard Evaluation
Windows License valid for 137 days
Build 20348.fe_release.210507-1500
1:43 PM
7/25/2026
```

M1-T6-AD-PowerShell-Domain.png

Description

Verified the Active Directory module and confirmed communication with the Active Directory domain.

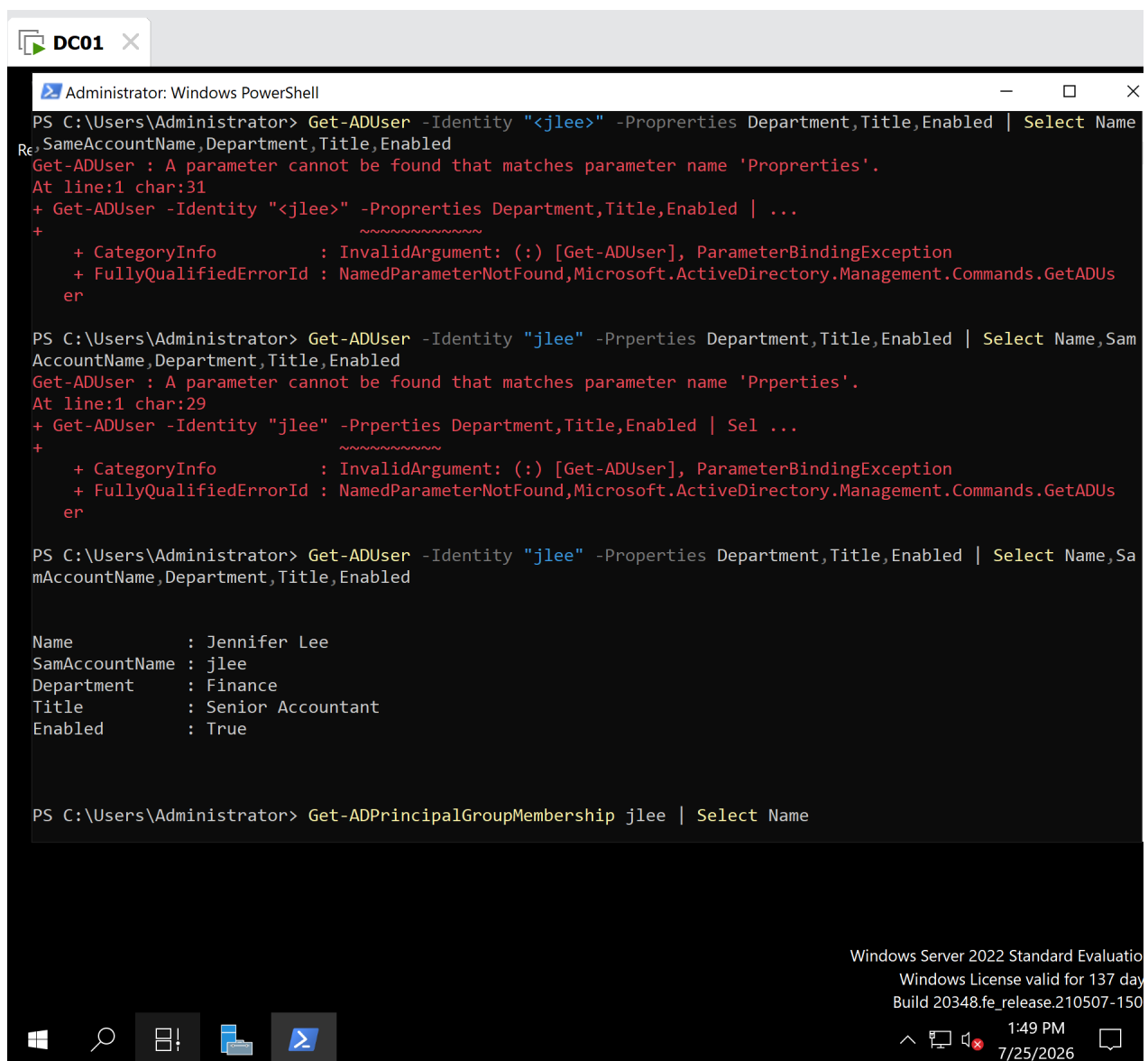
```
Administrator: Windows PowerShell
PS C:\Users\Administrator> Get-ADUser -Filter * -SearchBase "OU=Finance,OU=Departments,OU=BEN Industries,DC=benlab,DC=local" | Select Name
Name
----
Robert Walker
Jennifer Lee
Kevin Adams
Sophia Turner

PS C:\Users\Administrator> Get-ADUser -Identity "Jennifer Lee"
```

M1-T6-PowerShell-Finance-Users.png

Description

Retrieved all users located within the Finance Organizational Unit.



```
DC01 X
Administrator: Windows PowerShell
PS C:\Users\Administrator> Get-ADUser -Identity "<jlee>" -Properties Department,Title,Enabled | Select Name, SamAccountName, Department, Title, Enabled
Get-ADUser : A parameter cannot be found that matches parameter name 'Properties'.
At line:1 char:31
+ Get-ADUser -Identity "<jlee>" -Properties Department,Title,Enabled | ...
+ ~~~~~
+ CategoryInfo          : InvalidArgument: (:) [Get-ADUser], ParameterBindingException
+ FullyQualifiedErrorId : NamedParameterNotFound,Microsoft.ActiveDirectory.Management.Commands.GetADUser

PS C:\Users\Administrator> Get-ADUser -Identity "jlee" -Properties Department,Title,Enabled | Select Name, SamAccountName, Department, Title, Enabled
Get-ADUser : A parameter cannot be found that matches parameter name 'Properties'.
At line:1 char:29
+ Get-ADUser -Identity "jlee" -Properties Department,Title,Enabled | Sel ...
+ ~~~~~
+ CategoryInfo          : InvalidArgument: (:) [Get-ADUser], ParameterBindingException
+ FullyQualifiedErrorId : NamedParameterNotFound,Microsoft.ActiveDirectory.Management.Commands.GetADUser

PS C:\Users\Administrator> Get-ADUser -Identity "jlee" -Properties Department,Title,Enabled | Select Name, SamAccountName, Department, Title, Enabled

Name           : Jennifer Lee
SamAccountName : jlee
Department     : Finance
Title          : Senior Accountant
Enabled        : True

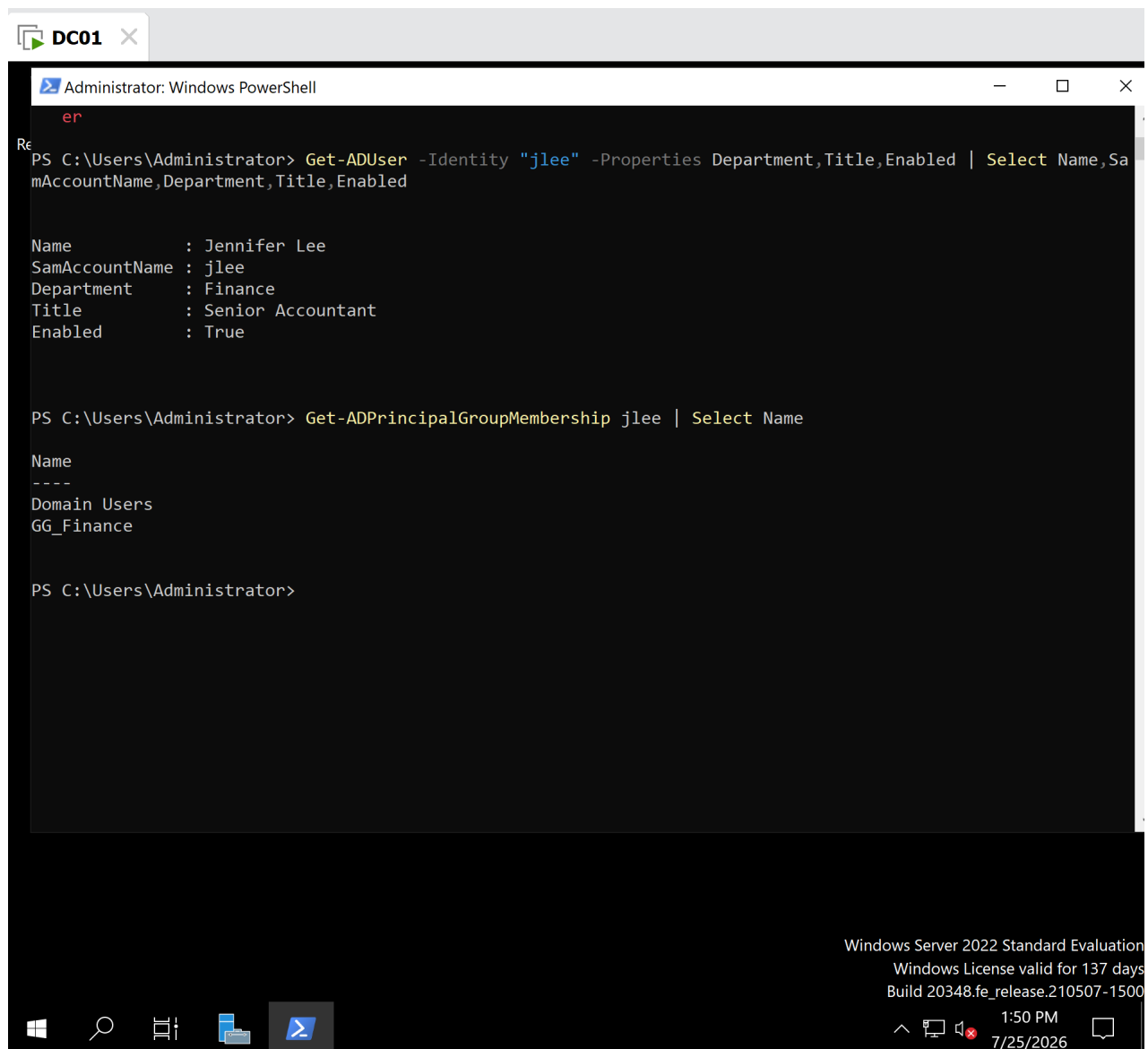
PS C:\Users\Administrator> Get-ADPrincipalGroupMembership jlee | Select Name

Windows Server 2022 Standard Evaluation
Windows License valid for 137 day
Build 20348.fe_release.210507-150
1:49 PM
7/25/2026
```

M1-T6-PowerShell-User-Properties.png

Description

Displayed Jennifer Lee's account information, including SamAccountName, department, job title, and account status.



```
er
Re
PS C:\Users\Administrator> Get-ADUser -Identity "jlee" -Properties Department,Title,Enabled | Select Name,Sa
mAccountName,Department,Title,Enabled

Name           : Jennifer Lee
SamAccountName  : jlee
Department     : Finance
Title          : Senior Accountant
Enabled        : True

PS C:\Users\Administrator> Get-ADPrincipalGroupMembership jlee | Select Name

Name
----
Domain Users
GG_Finance

PS C:\Users\Administrator>
```

M1-T6-PowerShell-GroupMembership.png

Description

Retrieved Jennifer Lee's Active Directory group memberships using PowerShell.

Key Concepts Learned

- The **Active Directory PowerShell module** extends PowerShell with cmdlets for administering Active Directory.
- `Import-Module ActiveDirectory` loads the Active Directory cmdlets into the current session.

- `Get-ADDomain` verifies domain configuration.
 - `Get-ADUser` retrieves Active Directory user objects.
 - `-Filter` specifies which objects should be returned.
 - `-SearchBase` limits the search to a specific Organizational Unit.
 - **SamAccountName** is the unique legacy Windows logon name commonly used by administrators.
 - `-Identity` works best with unique identifiers such as SamAccountName, Distinguished Name, GUID, or SID.
 - `Get-ADPrincipalGroupMembership` quickly identifies the security groups a user belongs to.
 - **GG** stands for **Global Group**, which is commonly used to group users by department or role.
-

Troubleshooting Performed

During this task, the following issues were encountered and resolved:

Issue 1

`Get-ADUser -Identity "Jennifer Lee"` returned an error because the **Display Name** is not the preferred identifier for the `-Identity` parameter.

Resolution

Identified the user's **SamAccountName (jlee)** and successfully queried the account.

Issue 2

PowerShell returned:

A parameter cannot be found that matches parameter name 'Prperties'

Cause

A typographical error in the parameter name.

Resolution

Corrected the spelling to:

-Properties

and the command executed successfully.

Lessons Learned

This task demonstrated how PowerShell complements graphical administration tools by providing faster, scriptable, and repeatable methods for managing Active Directory. Rather than manually navigating through administrative consoles, PowerShell enables administrators to retrieve user information, verify account status, inspect group memberships, and automate repetitive tasks using concise commands. Learning to use **SamAccountName**, understand **Global Groups (GG)**, and troubleshoot common syntax errors are foundational skills for Windows Server administration.

Administrator's Notes

Enterprise Best Practice: PowerShell is the preferred tool for bulk administration, automation, and reporting in enterprise Windows environments. While GUI tools such as ADUC and ADAC are excellent for interactive management, PowerShell is commonly used to create users, reset passwords, unlock accounts, manage group memberships, generate reports, and automate repetitive administrative tasks. A Windows Systems Administrator should be comfortable using both GUI-based management and PowerShell depending on the situation.

Task 7 – Active Directory User Lifecycle Management

Objective

Learn how to manage the complete lifecycle of an Active Directory user account using Windows PowerShell. This task demonstrates how to create a user account, verify its creation, assign group membership, verify permissions, remove the account, and confirm its deletion.

Background

User account management is one of the most common responsibilities of Windows Systems Administrators. While Active Directory Users and Computers (ADUC) provides a graphical interface for managing users, PowerShell allows administrators to perform these tasks faster, more consistently, and at scale.

Organizations often use PowerShell to automate employee onboarding, department transfers, security group assignments, and account deprovisioning.

Systems Used

Component	Value
Server	DC01
Operating System	Windows Server 2022
Domain	benlab.local
Tool	Windows PowerShell
Active Directory Module	ActiveDirectory
Test User	Olivia Parker (Temporary Account)

Steps Performed

Step 1 – Create a New Active Directory User

Created a temporary Finance employee account using the `New-ADUser` cmdlet.

Key account information included:

- Display Name: Olivia Parker
 - SamAccountName: oparker
 - User Principal Name (UPN): oparker@benlab.local
 - Department: Finance
 - Title: Financial Analyst
 - Organizational Unit: Finance OU
 - Enabled account with an initial password
-

Step 2 – Verify User Creation

Verified that the account was successfully created using:

```
Get-ADUser oparker
```

Confirmed:

- Account enabled
 - Correct SamAccountName
 - Correct UPN
 - Correct Organizational Unit
 - User created successfully
-

Step 3 – Add User to Finance Security Group

Added Olivia Parker to the Finance Global Security Group using:

```
Add-ADGroupMember `
```

```
-Identity "GG_Finance" `
```

```
-Members oparker
```

The command completed successfully without returning any errors.

Step 4 – Verify Group Membership

Verified Olivia Parker's group memberships using:

```
Get-ADPrincipalGroupMembership oparker |
```

```
Select Name
```

Returned:

- Domain Users
- GG_Finance

This confirmed the account was successfully added to the Finance security group.

Step 5 – Remove Temporary User

Deleted the temporary lab account using:

```
Remove-ADUser oparker
```

PowerShell requested confirmation before deleting the account.

Confirmed the deletion by selecting:

Step 6 – Verify User Deletion

Verified that the account had been removed by running:

```
Get-ADUser oparker
```

PowerShell returned:

```
Cannot find an object with identity: 'oparker'
```

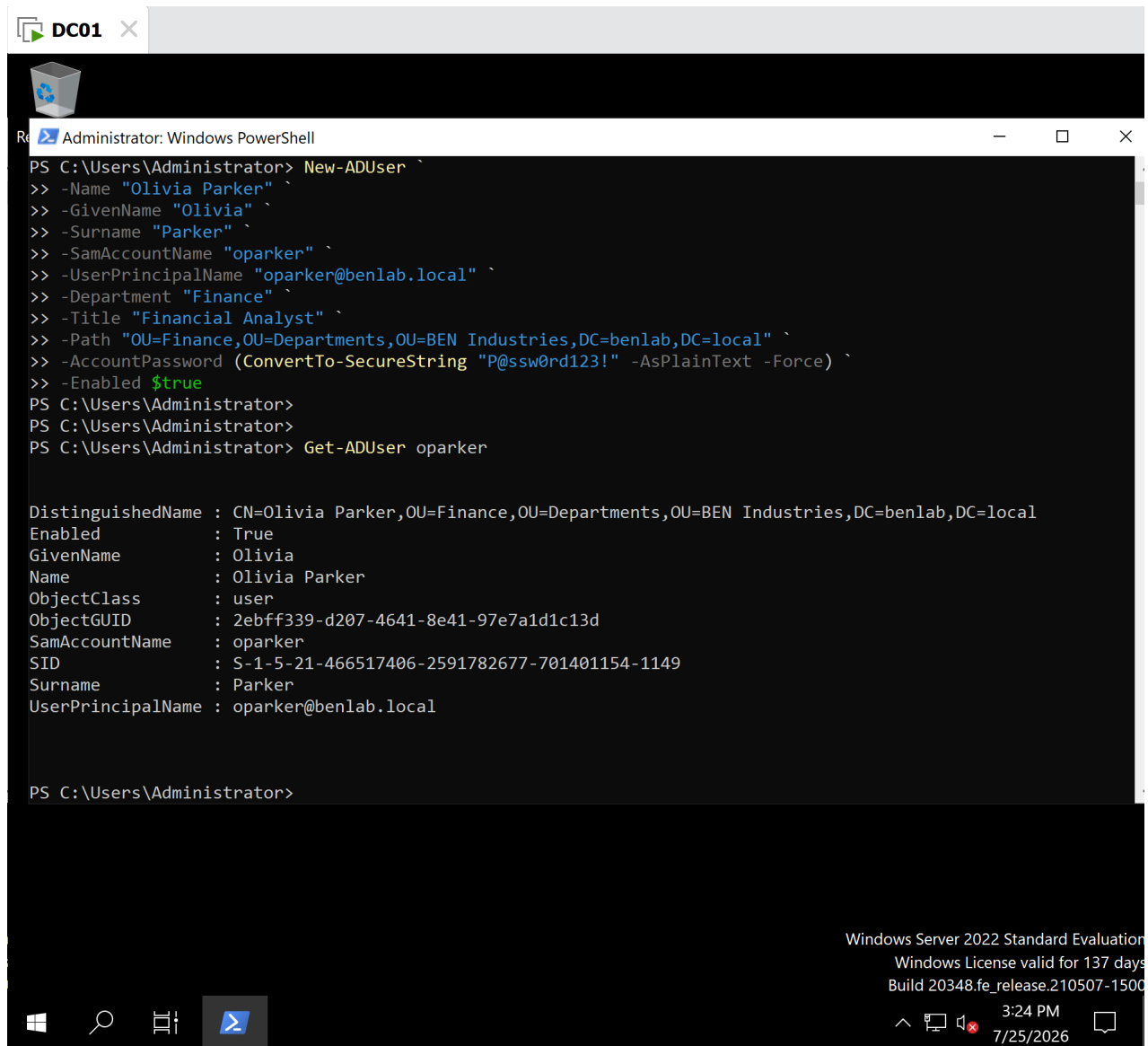
This confirmed that the account had been successfully removed from Active Directory.

Verification

The following administrative tasks were successfully completed:

- Created a new Active Directory user
 - Verified user account creation
 - Added the user to the Finance security group
 - Verified group membership
 - Deleted the temporary user account
 - Confirmed the account no longer existed
-

Screenshots



The screenshot shows a Windows PowerShell window titled "Administrator: Windows PowerShell" with a taskbar icon of a blue square with a white 'P'. The window is running commands to create a new Active Directory user. The commands are as follows:

```
PS C:\Users\Administrator> New-ADUser `
>> -Name "Olivia Parker" `
>> -GivenName "Olivia" `
>> -Surname "Parker" `
>> -SamAccountName "oparker" `
>> -UserPrincipalName "oparker@benlab.local" `
>> -Department "Finance" `
>> -Title "Financial Analyst" `
>> -Path "OU=Finance,OU=Departments,OU=BEN Industries,DC=benlab,DC=local" `
>> -AccountPassword (ConvertTo-SecureString "P@ssw0rd123!" -AsPlainText -Force) `
>> -Enabled $true
PS C:\Users\Administrator>
PS C:\Users\Administrator>
PS C:\Users\Administrator> Get-ADUser oparker
```

The output of the `Get-ADUser oparker` command is displayed below:

```
DistinguishedName : CN=Olivia Parker,OU=Finance,OU=Departments,OU=BEN Industries,DC=benlab,DC=local
Enabled           : True
GivenName        : Olivia
Name             : Olivia Parker
ObjectClass      : user
ObjectGUID       : 2ebff339-d207-4641-8e41-97e7a1d1c13d
SamAccountName   : oparker
SID              : S-1-5-21-466517406-2591782677-701401154-1149
Surname          : Parker
UserPrincipalName : oparker@benlab.local
```

The PowerShell prompt is now `PS C:\Users\Administrator>`.

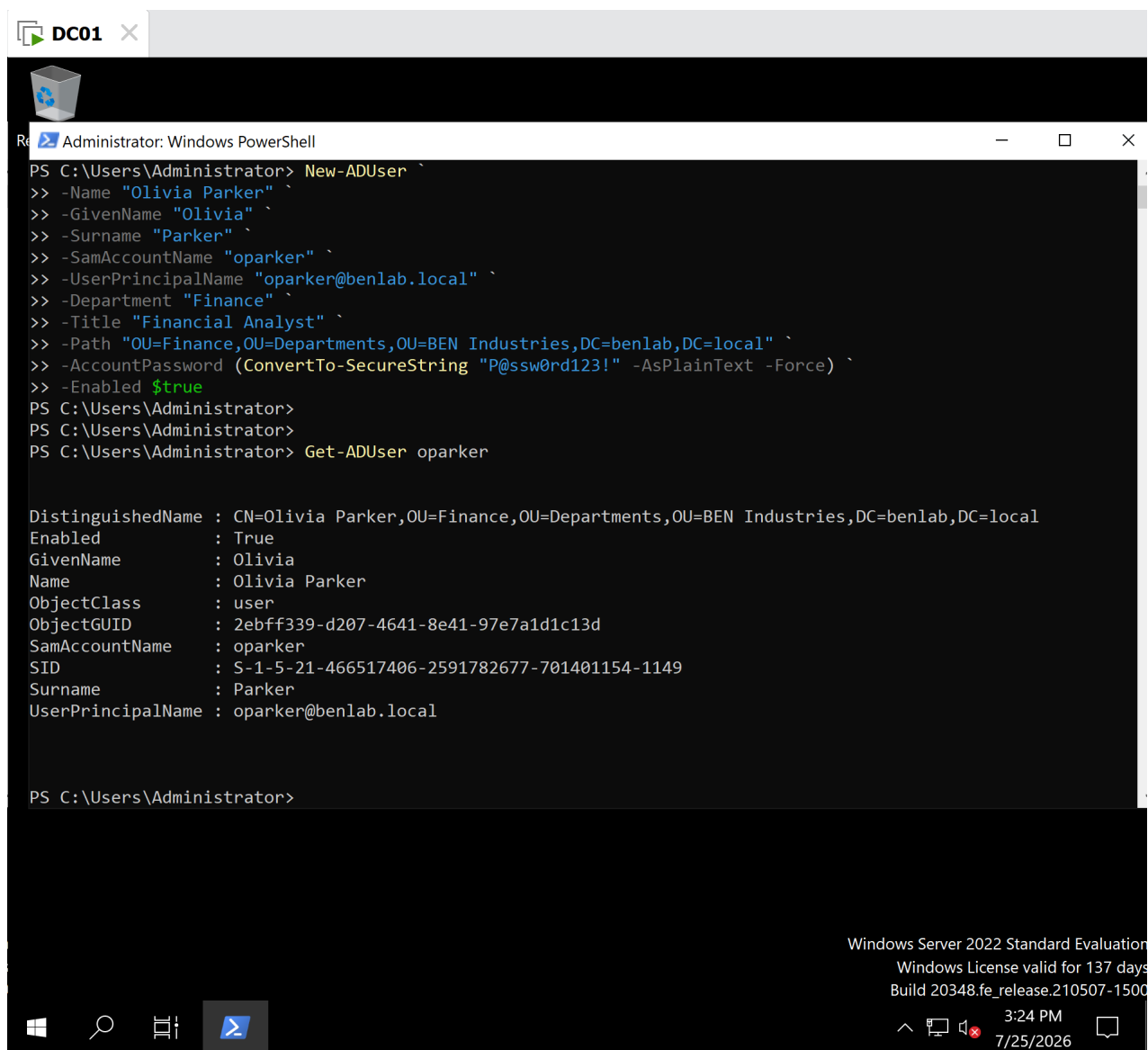
At the bottom right of the window, the following text is visible:

```
Windows Server 2022 Standard Evaluation
Windows License valid for 137 days
Build 20348.fe_release.210507-1500
3:24 PM
7/25/2026
```

M1-T7-New-ADUser . png

Description

Created the temporary Active Directory user account using the `New-ADUser` PowerShell cmdlet.



```
PS C:\Users\Administrator> New-ADUser `
>> -Name "Olivia Parker" `
>> -GivenName "Olivia" `
>> -Surname "Parker" `
>> -SamAccountName "oparker" `
>> -UserPrincipalName "oparker@benlab.local" `
>> -Department "Finance" `
>> -Title "Financial Analyst" `
>> -Path "OU=Finance,OU=Departments,OU=BEN Industries,DC=benlab,DC=local" `
>> -AccountPassword (ConvertTo-SecureString "P@ssw0rd123!" -AsPlainText -Force) `
>> -Enabled $true
PS C:\Users\Administrator>
PS C:\Users\Administrator>
PS C:\Users\Administrator> Get-ADUser oparker

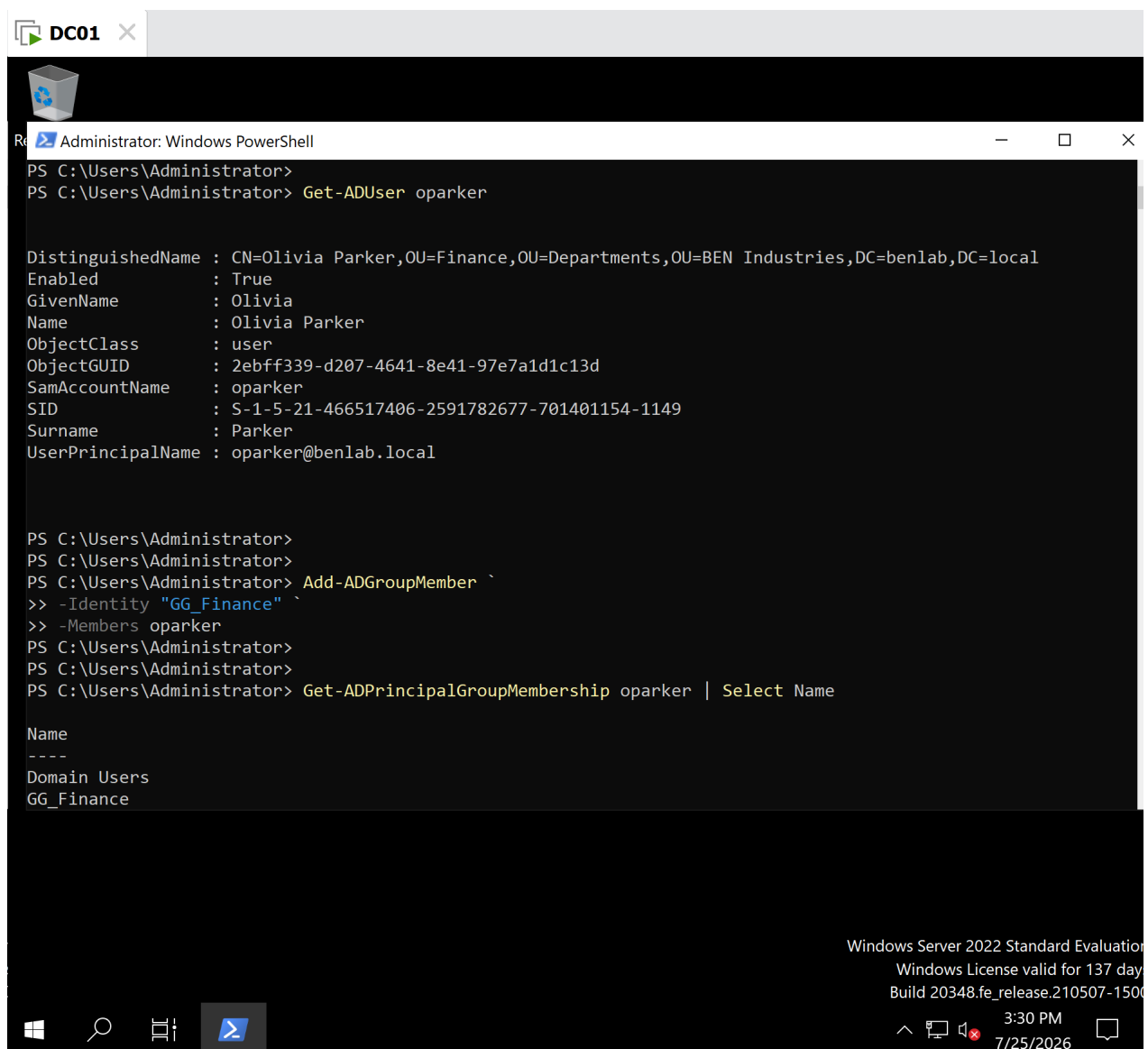
DistinguishedName : CN=Olivia Parker,OU=Finance,OU=Departments,OU=BEN Industries,DC=benlab,DC=local
Enabled           : True
GivenName        : Olivia
Name             : Olivia Parker
ObjectClass      : user
ObjectGUID       : 2ebff339-d207-4641-8e41-97e7a1d1c13d
SamAccountName   : oparker
SID              : S-1-5-21-466517406-2591782677-701401154-1149
Surname          : Parker
UserPrincipalName : oparker@benlab.local

PS C:\Users\Administrator>
```

M1-T7-Verify-User.png

Description

Verified the newly created user account using Get -ADUser.



```
DC01 x
Administrator: Windows PowerShell
PS C:\Users\Administrator>
PS C:\Users\Administrator> Get-ADUser oparker

DistinguishedName : CN=Olivia Parker,OU=Finance,OU=Departments,OU=BEN Industries,DC=benlab,DC=local
Enabled            : True
GivenName         : Olivia
Name              : Olivia Parker
ObjectClass       : user
ObjectGUID        : 2ebff339-d207-4641-8e41-97e7a1d1c13d
SamAccountName    : oparker
SID               : S-1-5-21-466517406-2591782677-701401154-1149
Surname           : Parker
UserPrincipalName : oparker@benlab.local

PS C:\Users\Administrator>
PS C:\Users\Administrator>
PS C:\Users\Administrator> Add-ADGroupMember `
>> -Identity "GG_Finance" `
>> -Members oparker
PS C:\Users\Administrator>
PS C:\Users\Administrator>
PS C:\Users\Administrator> Get-ADPrincipalGroupMembership oparker | Select Name

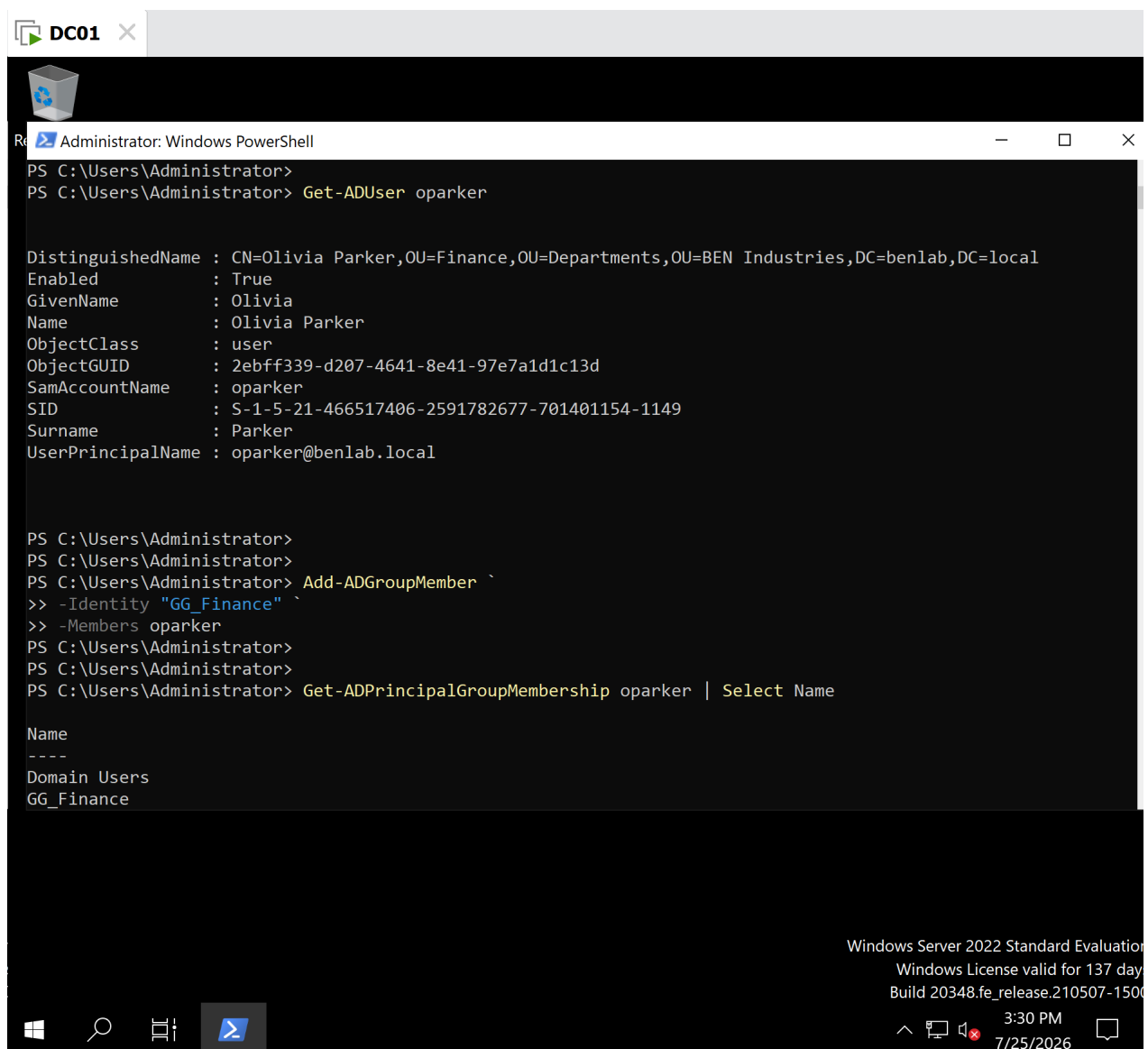
Name
----
Domain Users
GG_Finance

Windows Server 2022 Standard Evaluation
Windows License valid for 137 days
Build 20348.fe_release.210507-1500
3:30 PM
7/25/2026
```

M1-T7-Add-GroupMembership.png

Description

Added Olivia Parker to the **GG_Finance** security group using Add-ADGroupMember.



```
PS C:\Users\Administrator> Get-ADUser oparker

DistinguishedName : CN=Olivia Parker,OU=Finance,OU=Departments,OU=BEN Industries,DC=benlab,DC=local
Enabled            : True
GivenName         : Olivia
Name              : Olivia Parker
ObjectClass       : user
ObjectGUID        : 2ebff339-d207-4641-8e41-97e7a1d1c13d
SamAccountName    : oparker
SID               : S-1-5-21-466517406-2591782677-701401154-1149
Surname           : Parker
UserPrincipalName : oparker@benlab.local

PS C:\Users\Administrator>
PS C:\Users\Administrator>
PS C:\Users\Administrator> Add-ADGroupMember `
>> -Identity "GG_Finance" `
>> -Members oparker
PS C:\Users\Administrator>
PS C:\Users\Administrator> Get-ADPrincipalGroupMembership oparker | Select Name

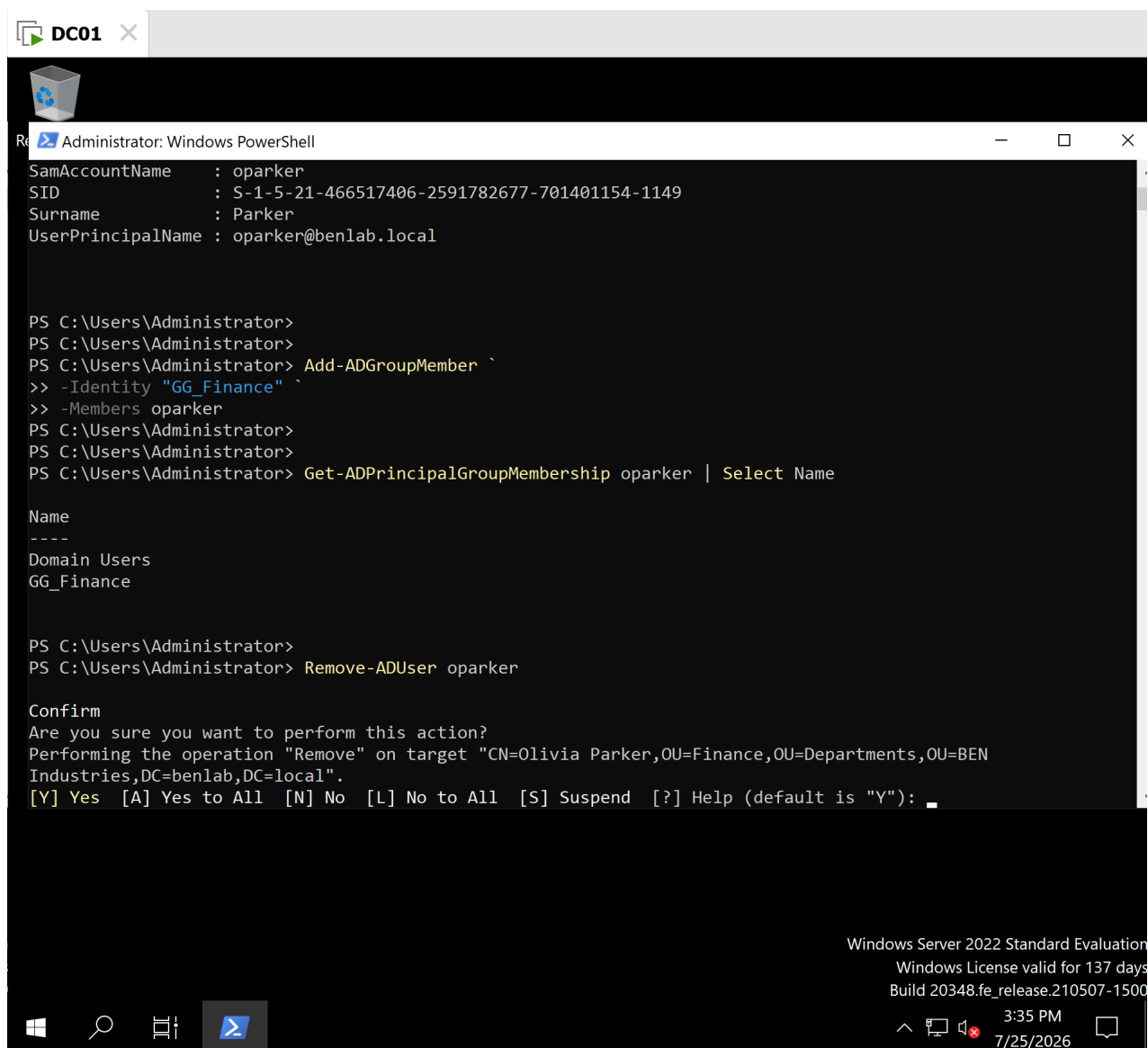
Name
----
Domain Users
GG_Finance

Windows Server 2022 Standard Evaluation
Windows License valid for 137 days
Build 20348.fe_release.210507-1500
3:30 PM
7/25/2026
```

M1-T7-Verify-GroupMembership.png

Description

Verified the user's membership in the **GG_Finance** security group using `Get-ADPrincipalGroupMembership`.



```
SamAccountName : oparker
SID             : S-1-5-21-466517406-2591782677-701401154-1149
Surname         : Parker
UserPrincipalName : oparker@benlab.local

PS C:\Users\Administrator>
PS C:\Users\Administrator>
PS C:\Users\Administrator> Add-ADGroupMember `
>> -Identity "GG_Finance" `
>> -Members oparker
PS C:\Users\Administrator>
PS C:\Users\Administrator>
PS C:\Users\Administrator> Get-ADPrincipalGroupMembership oparker | Select Name

Name
----
Domain Users
GG_Finance

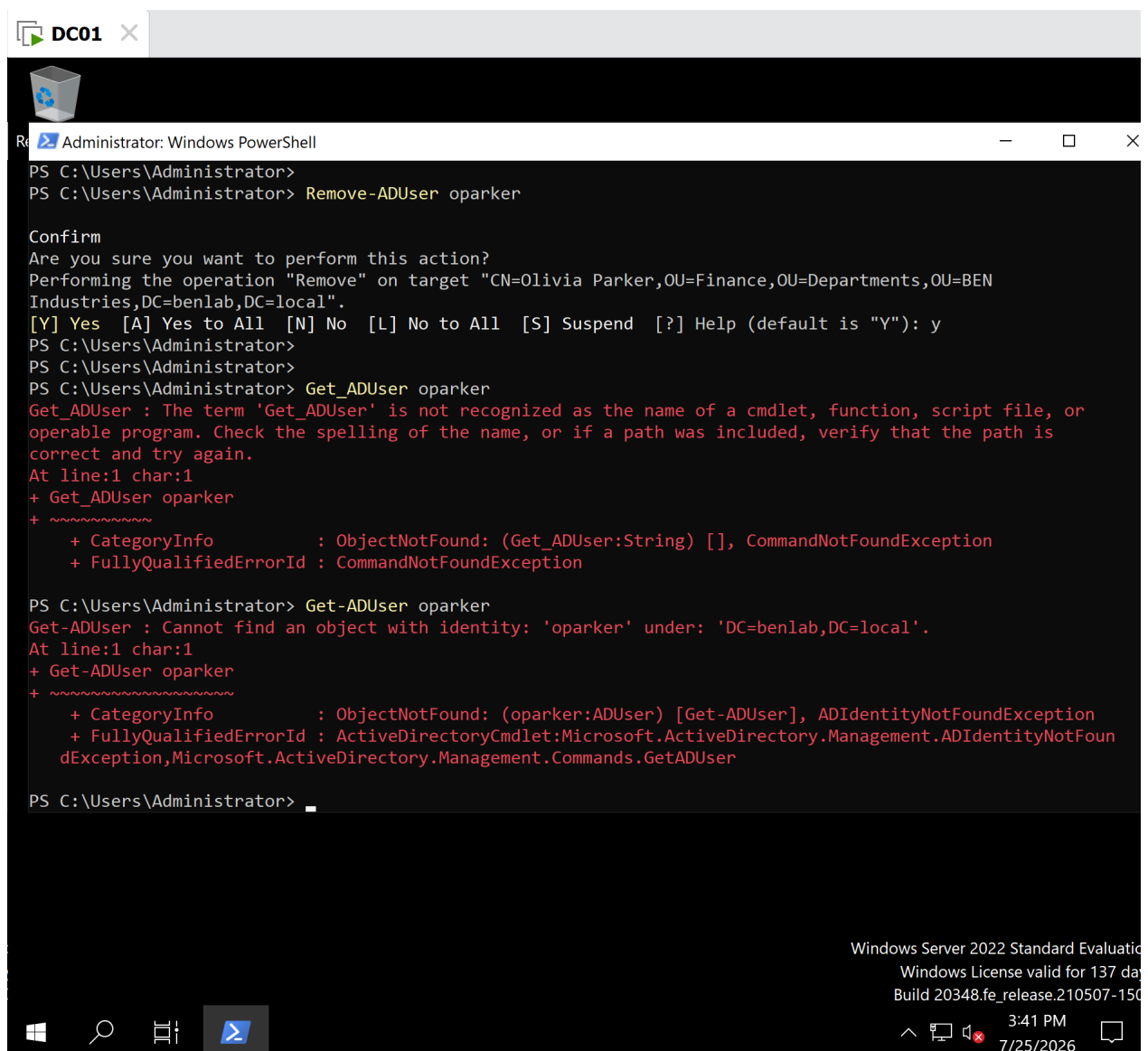
PS C:\Users\Administrator>
PS C:\Users\Administrator> Remove-ADUser oparker

Confirm
Are you sure you want to perform this action?
Performing the operation "Remove" on target "CN=Olivia Parker,OU=Finance,OU=Departments,OU=BEN
Industries,DC=benlab,DC=local".
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "Y"): 
```

M1-T7-Remove-User .png

Description

Removed the temporary Active Directory user account using `Remove-ADUser` and confirmed the deletion.



```
PS C:\Users\Administrator>
PS C:\Users\Administrator> Remove-ADUser oparker

Confirm
Are you sure you want to perform this action?
Performing the operation "Remove" on target "CN=Olivia Parker,OU=Finance,OU=Departments,OU=BEN
Industries,DC=benlab,DC=local".
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "Y"): y
PS C:\Users\Administrator>
PS C:\Users\Administrator> Get_ADUser oparker
Get_ADUser : The term 'Get_ADUser' is not recognized as the name of a cmdlet, function, script file, or
operable program. Check the spelling of the name, or if a path was included, verify that the path is
correct and try again.
At line:1 char:1
+ Get_ADUser oparker
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (Get_ADUser:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\Administrator> Get-ADUser oparker
Get-ADUser : Cannot find an object with identity: 'oparker' under: 'DC=benlab,DC=local'.
At line:1 char:1
+ Get-ADUser oparker
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (oparker:ADUser) [Get-ADUser], ADIdentityNotFoundException
+ FullyQualifiedErrorId : ActiveDirectoryCmdlet:Microsoft.ActiveDirectory.Management.ADIdentityNotFoun
dException,Microsoft.ActiveDirectory.Management.Commands.GetADUser

PS C:\Users\Administrator>
```

M1-T7-Verify-Deletion.png

Description

Verified the account no longer existed by running `Get -ADUser`, which returned the expected "Cannot find an object with identity" message.

Key Concepts Learned

- `New-ADUser` creates Active Directory user accounts.
- `Get -ADUser` retrieves and verifies user information.

- `Add-ADGroupMember` adds users to security groups.
 - `Get-ADPrincipalGroupMembership` displays a user's group memberships.
 - `Remove-ADUser` removes user accounts from Active Directory.
 - `SamAccountName` is commonly used as the unique identifier in PowerShell.
 - PowerShell prompts for confirmation before performing destructive actions such as deleting user accounts.
 - Successful PowerShell commands often complete silently and return to the prompt without displaying a success message.
-

Troubleshooting Performed

Issue 1 – Missing Backticks

The command terminated early because a line continuation backtick (```) was omitted.

Resolution

Added the required backtick at the end of each line except the final line.

Issue 2 – Missing Parameter Hyphens

Some parameters were entered without the required leading hyphen (for example, `Title` instead of `-Title`).

Resolution

Corrected all parameter names to include the leading hyphen.

Issue 3 – Duplicate User Principal Name (UPN)

Attempting to create **Emily Carter** generated an error because the account already existed in Active Directory.

Resolution

Created a new temporary account named **Olivia Parker** using a unique SamAccountName and UPN.

Issue 4 – Cmdlet Typographical Error

Initially entered:

```
Get_ADUser
```

instead of:

```
Get-ADUser
```

Resolution

Corrected the cmdlet by replacing the underscore (_) with a hyphen (-).

Lessons Learned

This task demonstrated the complete lifecycle of an Active Directory user account using PowerShell. It reinforced the importance of accurate PowerShell syntax, including proper parameter formatting, line continuation characters, and command verification. It also highlighted the value of interpreting PowerShell error messages to troubleshoot syntax issues, duplicate objects, and verification tasks. Managing users through PowerShell provides a faster and more scalable approach than relying solely on graphical administration tools.

Administrator's Notes

Enterprise Best Practice: Many organizations automate user provisioning and deprovisioning with PowerShell. Administrators commonly use scripts to create user accounts, assign security groups, configure department attributes, and remove accounts during employee offboarding. Verifying each administrative action with cmdlets such as `Get-ADUser` and `Get-ADPrincipalGroupMembership` helps ensure changes were applied correctly before moving to production systems.

Task 8 – Active Directory Group Lifecycle Management

Objective

Learn how to manage Active Directory security groups using Windows PowerShell by creating a security group, verifying its configuration, adding users, verifying membership, removing users, deleting the group, and confirming its removal from Active Directory.

Background

Security groups are fundamental to access management in Active Directory. Rather than assigning permissions directly to individual users, administrators assign permissions to security groups and then add or remove users from those groups. This simplifies administration, improves security, and supports the Principle of Least Privilege.

PowerShell enables administrators to efficiently create, modify, and remove groups while automating repetitive administrative tasks.

Systems Used

Component	Value
Server	DC01
Operating System	Windows Server 2022
Domain	benlab.local
Tool	Windows PowerShell
Active Directory Module	ActiveDirectory
Temporary Group	GG_ProjectAlpha

Steps Performed

Step 1 – Create a Security Group

Created a temporary Global Security Group named **GG_ProjectAlpha** using the `New-ADGroup` cmdlet.

Configuration included:

- Group Name: GG_ProjectAlpha
- SamAccountName: GG_ProjectAlpha

- Group Scope: Global
 - Group Category: Security
 - Location: Groups OU
-

Step 2 – Verify Group Creation

Verified the group using:

```
Get-ADGroup GG_ProjectAlpha
```

Confirmed:

- Group exists
 - Security group
 - Global scope
 - Correct Organizational Unit
 - Correct SamAccountName
-

Step 3 – Add Users to the Group

Added existing Active Directory users to the temporary group:

- Jennifer Lee (jlee)
- Kevin Adams (kadams)

Command used:

```
Add-ADGroupMember `
-Identity "GG_ProjectAlpha" `
-Members jlee,kadams
```

The command completed successfully without errors.

Step 4 – Verify Group Membership

Verified the group membership using:

```
Get-ADGroupMember GG_ProjectAlpha |
Select Name,ObjectClass
```

Returned:

- Jennifer Lee

- Kevin Adams

Both objects were verified as Active Directory user accounts.

Step 5 – Remove a Group Member

Simulated a project change by removing Kevin Adams from the group.

Command used:

```
Remove-ADGroupMember `
-Identity "GG_ProjectAlpha" `
-Members kadams
```

PowerShell requested confirmation before removing the user.

Step 6 – Verify Updated Membership

Verified the updated membership using:

```
Get-ADGroupMember GG_ProjectAlpha |
Select Name
```

Confirmed only:

- Jennifer Lee

remained a member of the group.

Step 7 – Remove the Temporary Group

Deleted the temporary security group using:

```
Remove-ADGroup GG_ProjectAlpha
```

PowerShell requested confirmation before deleting the group.

Step 8 – Verify Group Deletion

Verified the deletion by running:

```
Get-ADGroup GG_ProjectAlpha
```

PowerShell returned:

```
Cannot find an object with identity: 'GG_ProjectAlpha'
```

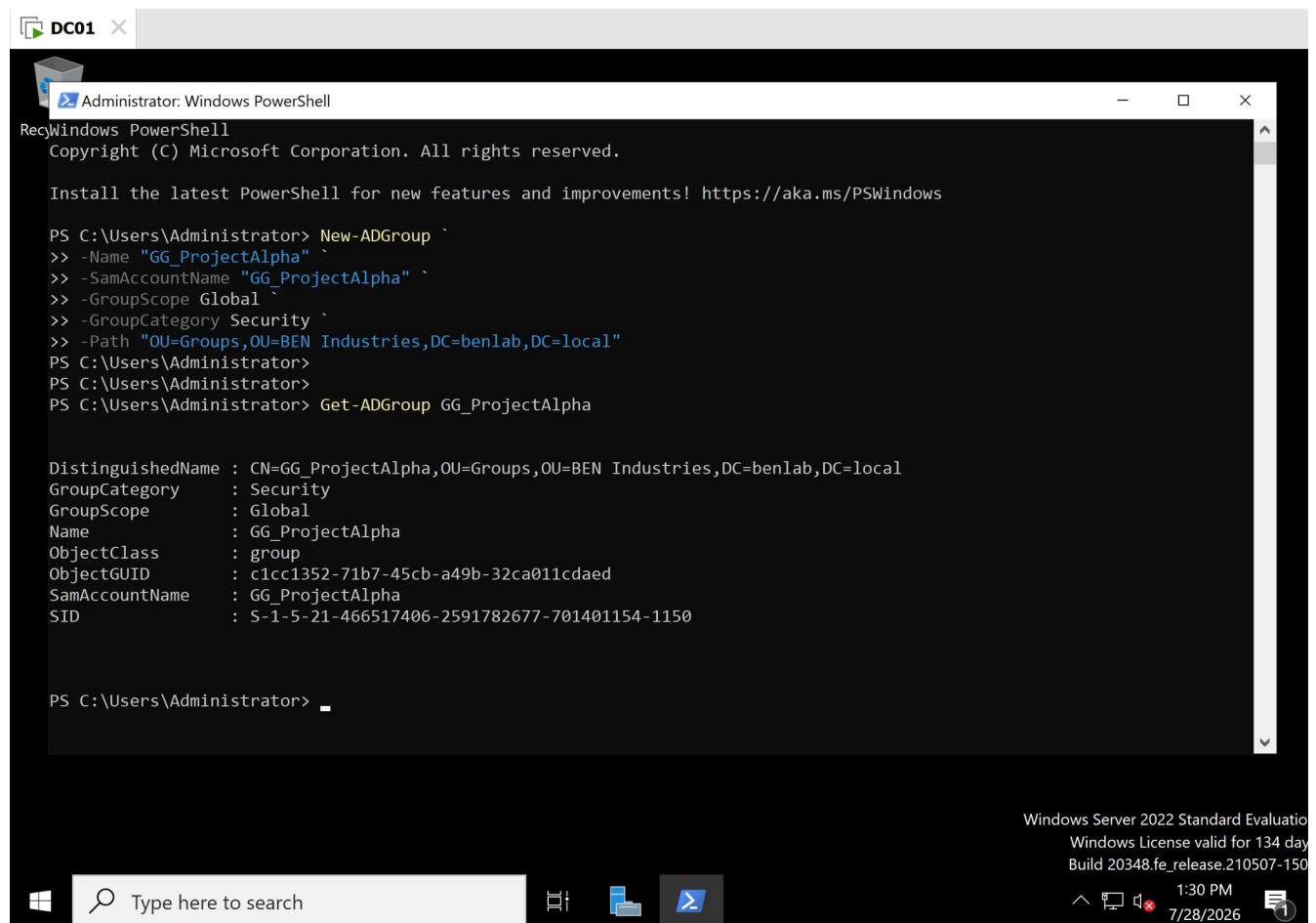
This confirmed the group had been successfully removed from Active Directory.

Verification

The following administrative tasks were successfully completed:

- Created a Global Security Group
- Verified the group configuration
- Added users to the group
- Verified group membership
- Removed a user from the group
- Verified updated membership
- Deleted the temporary group
- Confirmed the group no longer existed

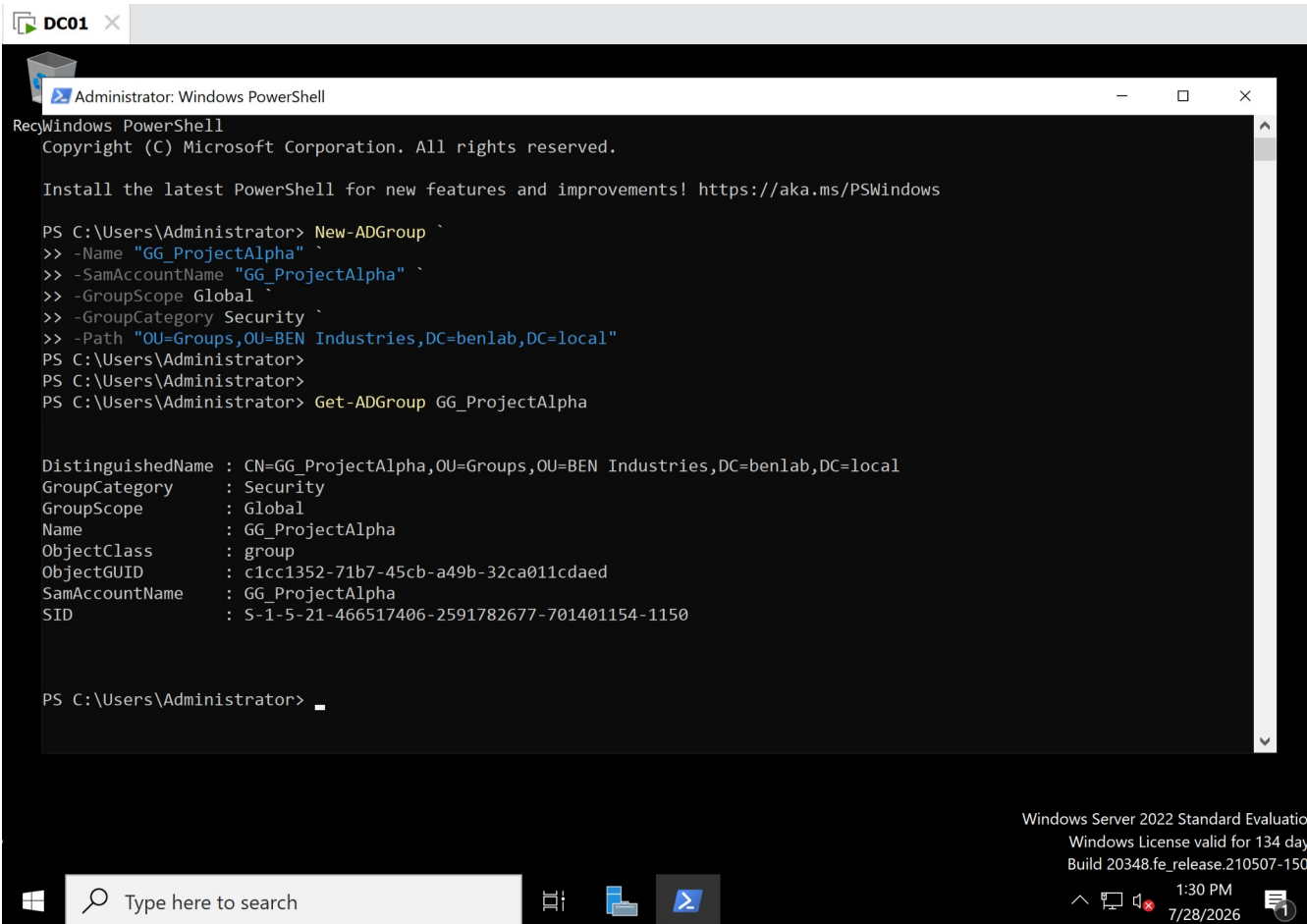
Screenshots



M1 - T8 - New-ADGroup.png

Description

Created the temporary security group **GG_ProjectAlpha** using the New-ADGroup PowerShell cmdlet.



The screenshot shows a Windows PowerShell terminal window titled "Administrator: Windows PowerShell". The window displays the following commands and output:

```
PS C:\Users\Administrator> New-ADGroup `
>> -Name "GG_ProjectAlpha" `
>> -SamAccountName "GG_ProjectAlpha" `
>> -GroupScope Global `
>> -GroupCategory Security `
>> -Path "OU=Groups,OU=BEN Industries,DC=benlab,DC=local"
PS C:\Users\Administrator>
PS C:\Users\Administrator>
PS C:\Users\Administrator> Get-ADGroup GG_ProjectAlpha

DistinguishedName : CN=GG_ProjectAlpha,OU=Groups,OU=BEN Industries,DC=benlab,DC=local
GroupCategory      : Security
GroupScope         : Global
Name               : GG_ProjectAlpha
ObjectClass        : group
ObjectGUID         : c1cc1352-71b7-45cb-a49b-32ca011cdaed
SamAccountName     : GG_ProjectAlpha
SID                : S-1-5-21-466517406-2591782677-701401154-1150

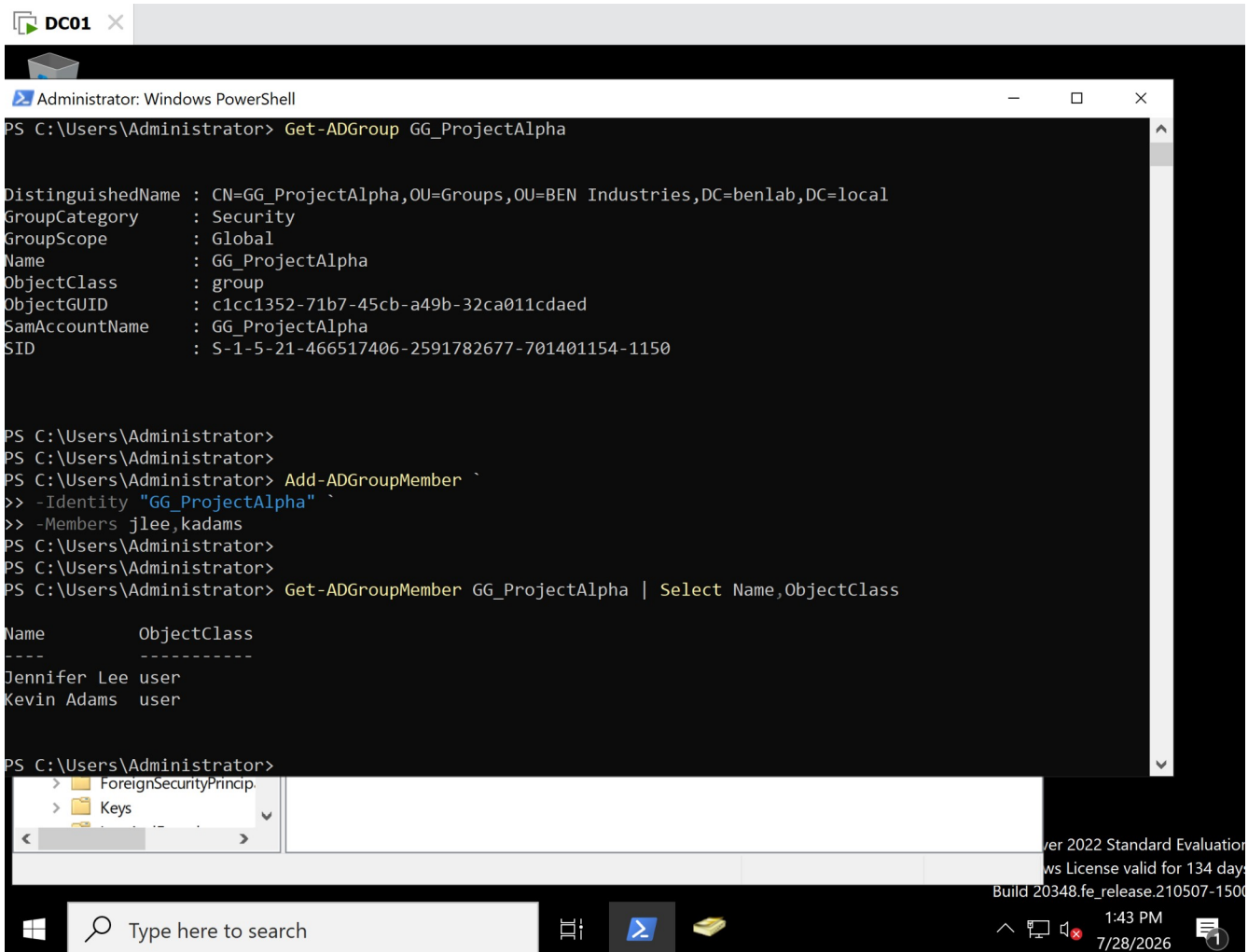
PS C:\Users\Administrator> _
```

The Windows taskbar at the bottom shows the search bar, task view, and several icons. The system tray on the right indicates "Windows Server 2022 Standard Evaluation", "Windows License valid for 134 day", "Build 20348.fe_release.210507-150", and the time "1:30 PM 7/28/2026".

M1-T8-Verify-Group.png

Description

Verified the group configuration, including Group Scope, Group Category, and Organizational Unit.



M1-T8-Add-GroupMembers.png

```
Administrator: Windows PowerShell
PS C:\Users\Administrator>
PS C:\Users\Administrator> Get-ADGroupMember GG_ProjectAlpha | Select Name

Name
----
Jennifer Lee

PS C:\Users\Administrator>
PS C:\Users\Administrator>
PS C:\Users\Administrator> Remove-ADGroup GG_ProjectAlpha

Confirm
Are you sure you want to perform this action?
Performing the operation "Remove" on target "CN=GG_ProjectAlpha,OU=Groups,OU=BEN
Industries,DC=benlab,DC=local".
[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "Y"): Y
PS C:\Users\Administrator>
PS C:\Users\Administrator>
PS C:\Users\Administrator> Get-ADGroup GG_ProjectAlpha
Get-ADGroup : Cannot find an object with identity: 'GG_ProjectAlpha' under: 'DC=benlab,DC=local'.
At line:1 char:1
+ Get-ADGroup GG_ProjectAlpha
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (GG_ProjectAlpha:ADGroup) [Get-ADGroup], ADIdentityNotFounExc
eption
+ FullyQualifiedErrorId : ActiveDirectoryCmdlet:Microsoft.ActiveDirectory.Management.ADIdentityNotFounE
xception,Microsoft.ActiveDirectory.Management.Commands.GetADGroup

PS C:\Users\Administrator>
```

M1-T8-Verify-GroupDeletion.png

Description

Verified the group and its members in Active Directory using the `Get-ADGroup` command, which returned the expected "Cannot find an object with identity: 'GG_ProjectAlpha' under: 'DC=benlab,DC=local'." message.

M1-T8-Verify-GroupMembers.png

Key Concepts Learned

Description

- `New-ADGroup` creates Active Directory security groups.
- `Get-ADGroup` retrieves and verifies group information.
- `Add-ADGroupMember` adds users to security groups.
- `Get-ADGroupMember` lists the members of a group.
- `Remove-ADGroupMember` removes users from security groups.
- `Remove-ADGroup` deletes security groups.

- Global Security Groups are commonly used to organize users for permission assignment.
 - PowerShell requests confirmation before destructive actions such as deleting groups.
-

Troubleshooting Performed

Issue 1 – Multi-line PowerShell Syntax

While entering PowerShell commands, proper use of backticks (` ) and parameter prefixes ( -`) was reinforced to ensure successful execution.

Resolution

Verified that each line ended with a backtick except the final line and confirmed all parameters included the required leading hyphen.

Issue 2 – Verification of Group Membership

Confirmed that users were successfully added and removed by using:

`Get -ADGroupMember`

rather than relying solely on command completion.

Resolution

Verified membership after every administrative change to ensure the expected results.

Lessons Learned

This task demonstrated the complete lifecycle of an Active Directory security group using PowerShell. It reinforced the importance of managing permissions through security groups instead of assigning permissions directly to users. The task also highlighted the value of verifying every administrative change using PowerShell cmdlets and maintaining a clean Active Directory environment by removing temporary objects after testing.

Administrator's Notes

Enterprise Best Practice: Organizations commonly assign permissions to security groups rather than individual users. Administrators add or remove users from groups as employees join projects, change departments, or leave the organization. PowerShell enables these changes to be performed consistently, efficiently, and at scale while providing quick verification through cmdlets such as `Get -ADGroup` and `Get -ADGroupMember`.

Task 9 – Bulk Active Directory User Management with PowerShell

Objective

Learn how to perform bulk administration of Active Directory user accounts using Windows PowerShell by identifying users based on a common attribute, modifying multiple user accounts simultaneously, verifying the changes, restoring the original configuration, and confirming the restoration.

Background

Enterprise environments often require administrators to update dozens or hundreds of user accounts at once. Rather than modifying each account individually through Active Directory Users and Computers (ADUC), PowerShell allows administrators to automate repetitive administrative tasks using pipelines and cmdlets.

Bulk administration improves efficiency, reduces administrative effort, and minimizes the risk of inconsistent configuration across user accounts.

Systems Used

Component	Value
Server	DC01
Operating System	Windows Server 2022
Domain	benlab.local
Tool	Windows PowerShell
Active Directory Module	ActiveDirectory
Department	Finance

Steps Performed

Step 1 – Display Finance Department Users

Retrieved all Finance department users along with their department and current job titles.

Command used:

```
Get-ADUser `
-Filter "Department -eq 'Finance'" `
```

```
-Properties Department,Title |  
Select Name,Department,Title
```

Verified the following users:

- Robert Walker
- Jennifer Lee
- Kevin Adams
- Sophia Turner

along with their original job titles.

Step 2 – Perform Bulk Job Title Update

Updated every user in the Finance department to the temporary title:

Senior Financial Analyst

Command used:

```
Get-ADUser `
-Filter "Department -eq 'Finance'" |
ForEach-Object {
    Set-ADUser $_ `
    -Title "Senior Financial Analyst"
}
```

PowerShell processed every matching user account through the pipeline and updated the **Title** attribute.

Step 3 – Verify Bulk Update

Verified the updated job titles by running:

```
Get-ADUser `
-Filter "Department -eq 'Finance'" `
-Properties Department,Title |
Select Name,Department,Title
```

Confirmed that every Finance user displayed:

- Senior Financial Analyst
-

Step 4 – Restore Original Job Titles

Returned each Finance user to their original job title using the `Set -ADUser` cmdlet.

Commands used:

```
Set-ADUser rwalker `
-Title "Finance Manager"
```

```
Set-ADUser jlee `
-Title "Senior Accountant"
```

```
Set-ADUser kadams `
-Title "Accountant"
```

```
Set-ADUser sturner `
-Title "Payroll Specialist"
```

Step 5 – Verify Restoration

Verified the original configuration using:

```
Get-ADUser `
-Filter "Department -eq 'Finance'" `
-Properties Department,Title |
Select Name,Department,Title
```

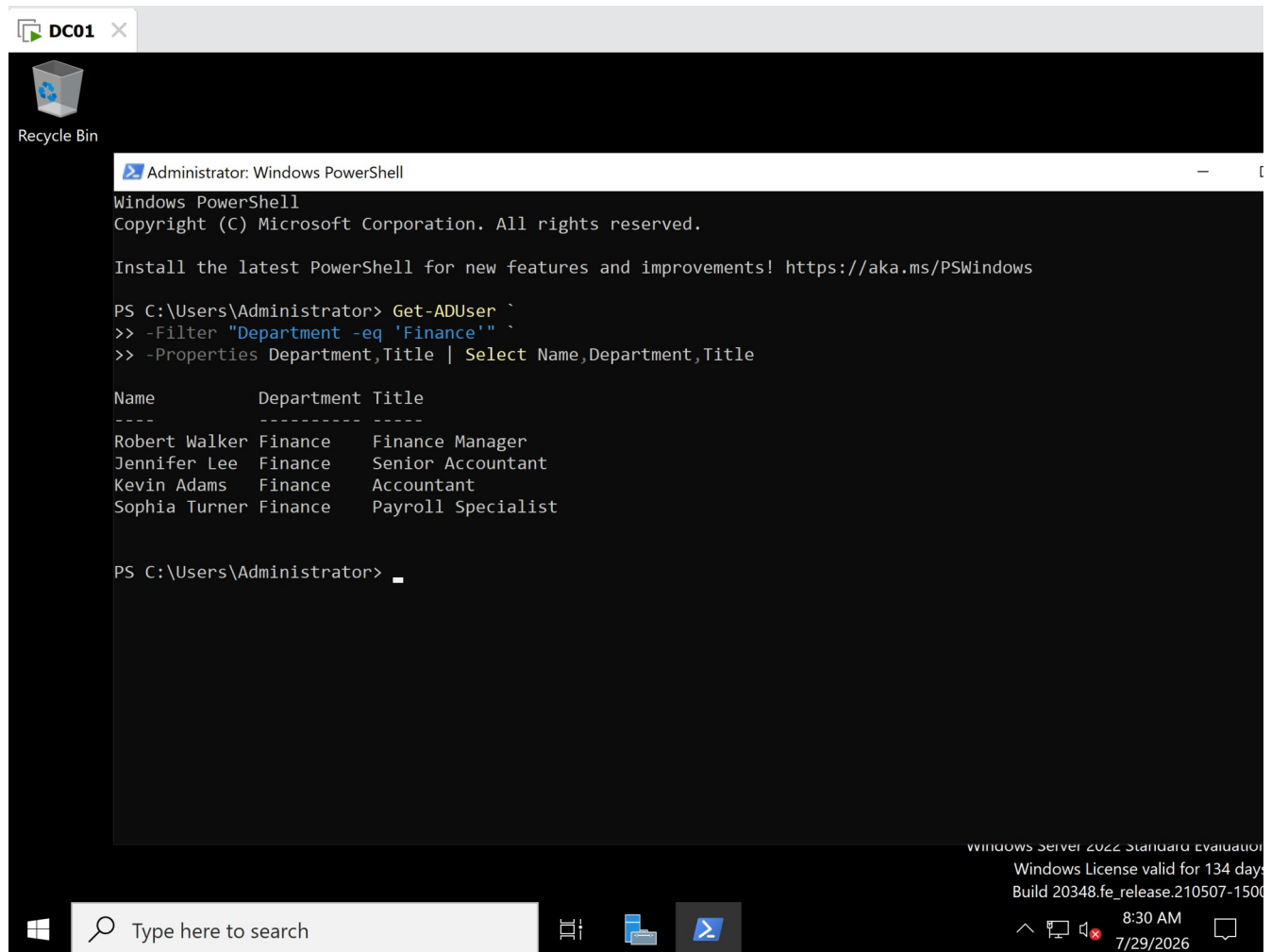
Confirmed the original titles had been successfully restored.

Verification

Successfully demonstrated:

- Retrieved users using the Department attribute.
- Used the `-eq` comparison operator to filter Active Directory objects.
- Performed a bulk modification using a PowerShell pipeline.
- Verified the updated user attributes.
- Restored the original configuration.
- Verified the restoration.

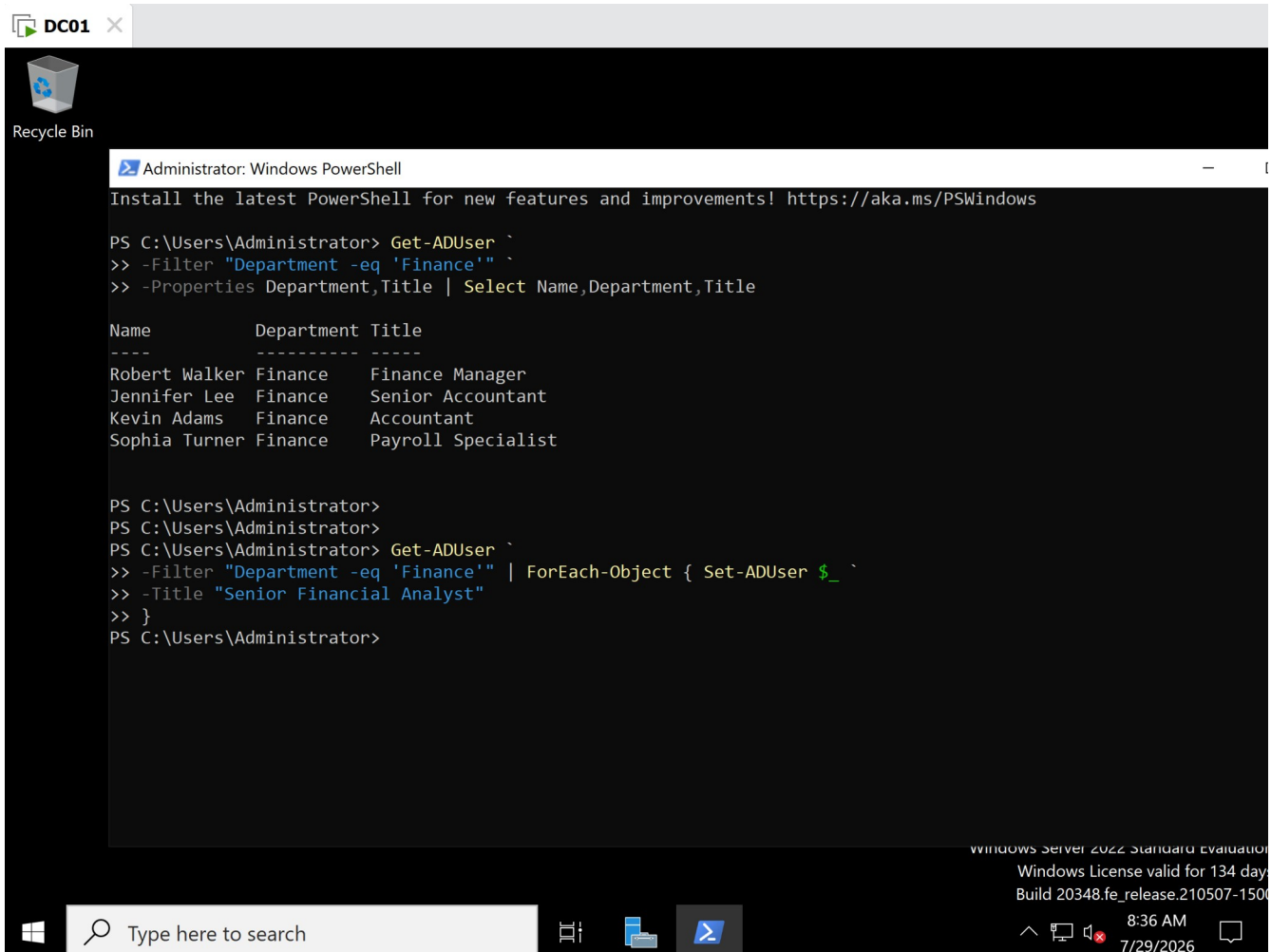
Screenshots



M1-T9-Current-Finance-Users.png

Description

Displayed all Finance department users and their original job titles before performing the bulk update.



M1-T9-Bulk-Update.png

Description

Executed a PowerShell pipeline using `Get-ADUser`, `ForEach-Object`, and `Set-ADUser` to update the Title attribute for every Finance department user.

DC01 x

Recycle Bin

```
Administrator: Windows PowerShell
PS C:\Users\Administrator>
PS C:\Users\Administrator>
PS C:\Users\Administrator> Get-UDUser `
>> -Filter "Department -eq 'Finance'" `
>> -Properties Department,Title |
>> Select Name,Department,Title
Get-UDUser : The term 'Get-UDUser' is not recognized as the name of a cmdlet, function, script file, or operabl
program. Check the spelling of the name, or if a path was included, verify that the path is correct and try aga
At line:1 char:1
+ Get-UDUser `
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (Get-UDUser:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\Administrator>
PS C:\Users\Administrator>
PS C:\Users\Administrator> Get-ADUser `
>> -Filter "Department -eq 'Finance'" `
>> -Properties Department,Title |
>> Select Name,Department,Title

Name            Department Title
-----
Robert Walker   Finance   Senior Financial Analyst
Jennifer Lee     Finance   Senior Financial Analyst
Kevin Adams     Finance   Senior Financial Analyst
Sophia Turner    Finance   Senior Financial Analyst

PS C:\Users\Administrator>
```

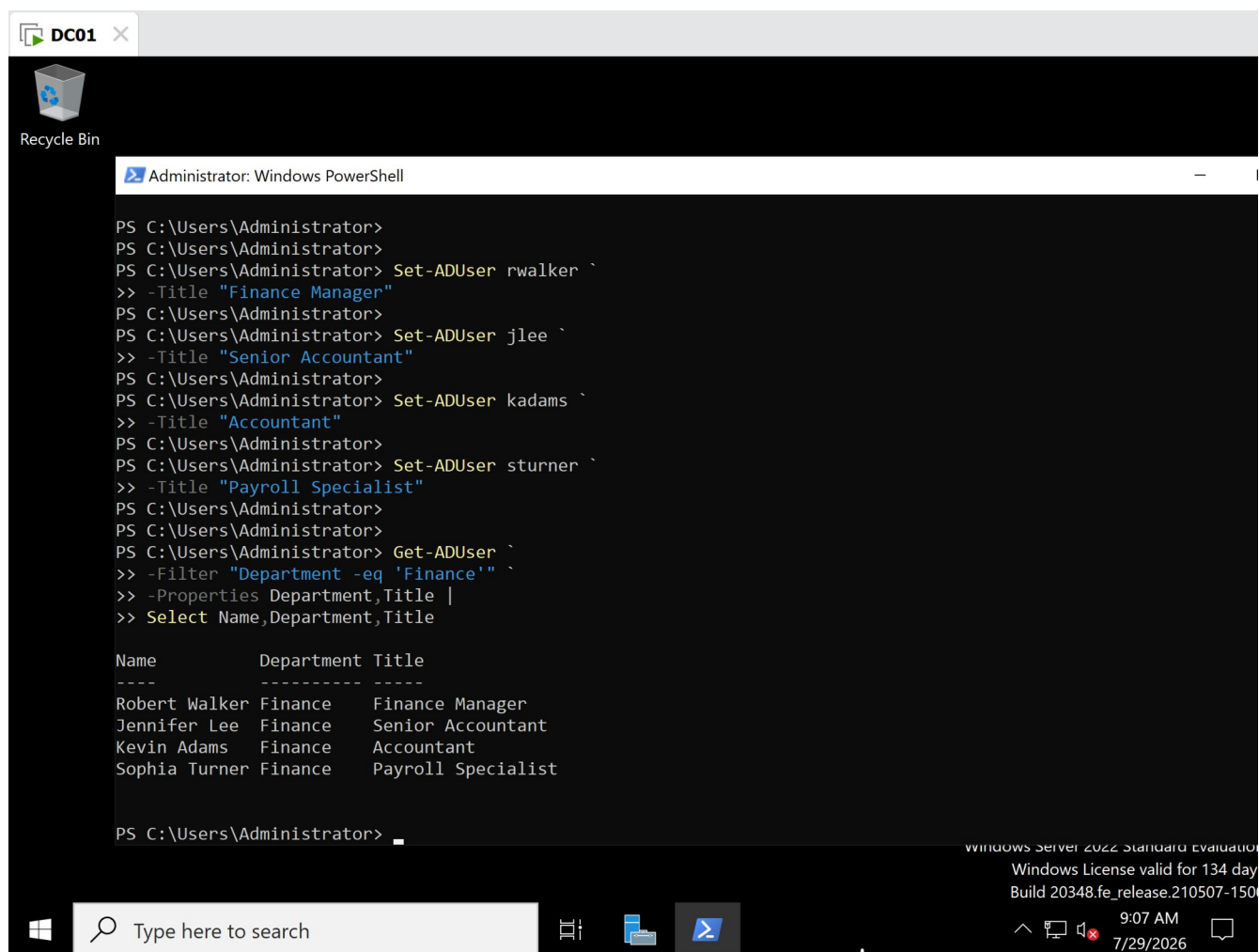
Windows Server 2022 Standard Evaluation
Windows License valid for 134 day
Build 20348.fe_release.210507-150

8:53 AM
7/29/2026

M1-T9-Verify-Bulk-Update.png

Description

Verified that every Finance department user had been updated to the temporary title **Senior Financial Analyst**.



```
PS C:\Users\Administrator>
PS C:\Users\Administrator>
PS C:\Users\Administrator> Set-ADUser rwalker `
>> -Title "Finance Manager"
PS C:\Users\Administrator>
PS C:\Users\Administrator> Set-ADUser jlee `
>> -Title "Senior Accountant"
PS C:\Users\Administrator>
PS C:\Users\Administrator> Set-ADUser kadams `
>> -Title "Accountant"
PS C:\Users\Administrator>
PS C:\Users\Administrator> Set-ADUser sturner `
>> -Title "Payroll Specialist"
PS C:\Users\Administrator>
PS C:\Users\Administrator>
PS C:\Users\Administrator> Get-ADUser `
>> -Filter "Department -eq 'Finance'" `
>> -Properties Department,Title |
>> Select Name,Department,Title

Name            Department Title
----            -
Robert Walker   Finance   Finance Manager
Jennifer Lee    Finance   Senior Accountant
Kevin Adams     Finance   Accountant
Sophia Turner   Finance   Payroll Specialist

PS C:\Users\Administrator>
```

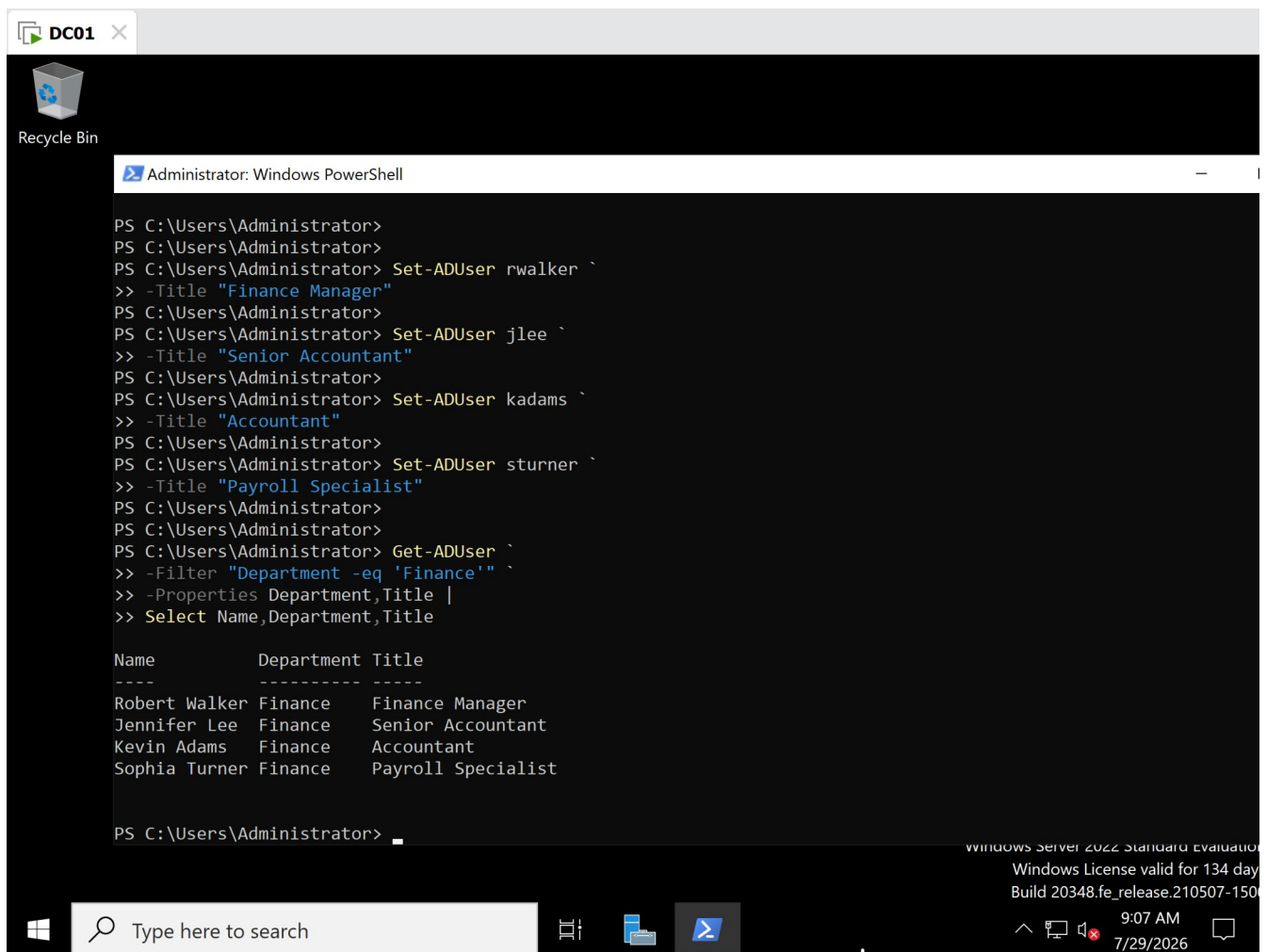
Windows Server 2022 Standard Evaluation
Windows License valid for 134 day
Build 20348.fe_release.210507-150

9:07 AM
7/29/2026

M1-T9-Restore-Titles.png

Description

Restored the original job titles for each Finance department user using individual Set -ADUser commands.



```
PS C:\Users\Administrator>
PS C:\Users\Administrator>
PS C:\Users\Administrator> Set-ADUser rwalker `
>> -Title "Finance Manager"
PS C:\Users\Administrator>
PS C:\Users\Administrator> Set-ADUser jlee `
>> -Title "Senior Accountant"
PS C:\Users\Administrator>
PS C:\Users\Administrator> Set-ADUser kadams `
>> -Title "Accountant"
PS C:\Users\Administrator>
PS C:\Users\Administrator> Set-ADUser sturner `
>> -Title "Payroll Specialist"
PS C:\Users\Administrator>
PS C:\Users\Administrator> Get-ADUser `
>> -Filter "Department -eq 'Finance'" `
>> -Properties Department,Title |
>> Select Name,Department,Title

Name            Department Title
----            -
Robert Walker   Finance   Finance Manager
Jennifer Lee     Finance   Senior Accountant
Kevin Adams      Finance   Accountant
Sophia Turner    Finance   Payroll Specialist

PS C:\Users\Administrator>
```

M1-T9-Verify-Restoration.png

Description

Verified that all Finance department users had been restored to their original job titles.

Key Concepts Learned

- `Get-ADUser` retrieves Active Directory user accounts.
- `Set-ADUser` modifies existing user attributes.
- `ForEach-Object` performs an action on every object passed through the pipeline.
- The PowerShell pipeline (`|`) passes output from one command directly to the next.
- The `-eq` operator compares two values for equality.
- Bulk administration is significantly faster than modifying users individually through the GUI.
- Administrative changes should always be verified after execution.

- Restoring a lab to its original state is a recommended practice after testing.
-

Troubleshooting Performed

Issue 1 – Incorrect Cmdlet Name

The command was initially entered as:

```
Get-UDUser
```

PowerShell returned a **CommandNotFoundException** because the cmdlet name was misspelled.

Resolution

Corrected the cmdlet to:

```
Get-ADUser
```

and successfully retrieved the Finance department users.

Issue 2 – Bulk Update Verification

After performing the bulk update, all Finance users displayed the temporary title:

Senior Financial Analyst

To preserve the integrity of the lab environment, the original job titles were restored and verified before concluding the task.

Lessons Learned

This task demonstrated the efficiency of PowerShell for bulk administration in Active Directory. Using filters, pipelines, and the `Set-ADUser` cmdlet allowed multiple user accounts to be updated simultaneously with minimal effort. The exercise also reinforced the importance of verifying administrative changes and restoring the lab environment to a known baseline after testing.

Administrator's Notes

Enterprise Best Practice: Large-scale user modifications should be performed using PowerShell rather than manually through graphical tools. Administrators should verify all changes immediately after execution and, when working in a lab or testing environment, restore the original configuration to maintain a consistent baseline for future exercises.

Task 10 – Enable and Disable Active Directory User Accounts Using PowerShell

Objective

Learn how to enable and disable Active Directory user accounts using PowerShell cmdlets while verifying account status before and after each administrative action.

Background

User accounts are frequently disabled in enterprise environments for security and administrative reasons, including employee leave, account compromise, temporary suspension, or offboarding. PowerShell allows administrators to perform these tasks quickly across one or many user accounts while maintaining consistency and automation.

Systems Used

- Windows Server 2022
 - Active Directory Domain Services (AD DS)
 - Windows PowerShell
 - Domain Controller: **DC01**
 - Domain: **benlab.local**
-

PowerShell Cmdlets Used

Verify Current Account Status

```
Get-ADUser kadams `
-Properties Enabled |
Select Name,Enabled
```

Disable the User Account

```
Disable-ADAccount kadams
```

Verify the Account is Disabled

```
Get-ADUser kadams `
```

-Properties Enabled |

Select Name,Enabled

Re-enable the User Account

Enable-ADAccount kadams

Verify the Account is Enabled Again

Get-ADUser kadams `

-Properties Enabled |

Select Name,Enabled

Verification

The PowerShell output confirmed the expected results:

Initial State

Kevin Adams

Enabled : True

↓

After Disabling

Kevin Adams

Enabled : False

↓

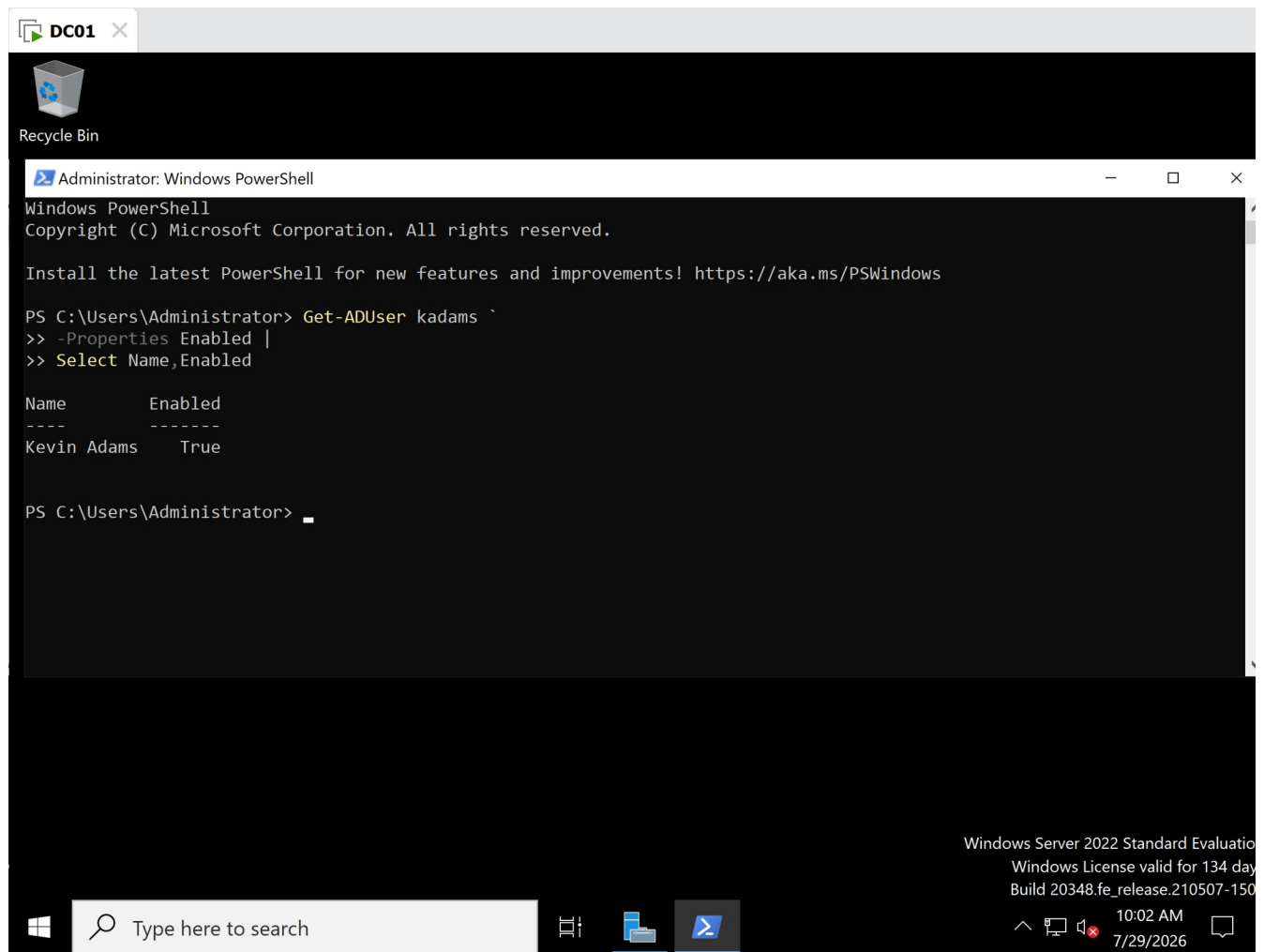
After Re-enabling

Kevin Adams

Enabled : True

This verified that the account status changed correctly after each administrative action.

Screenshots



Initial account status (Enabled = True)

DC01 x

Recycle Bin

Administrator: Windows PowerShell

```
PS C:\Users\Administrator> Get-ADUser kadams `
>> -Properties Enabled |
>> Select Name,Enabled

Name      Enabled
----      -
Kevin Adams    True

PS C:\Users\Administrator>
PS C:\Users\Administrator>
PS C:\Users\Administrator> Disable-ADAccount kadams
PS C:\Users\Administrator>
PS C:\Users\Administrator>
PS C:\Users\Administrator> Get-ADUser kadams `
>> -Properties Enabled |
>> Select Name,Enabled

Name      Enabled
----      -
Kevin Adams    False

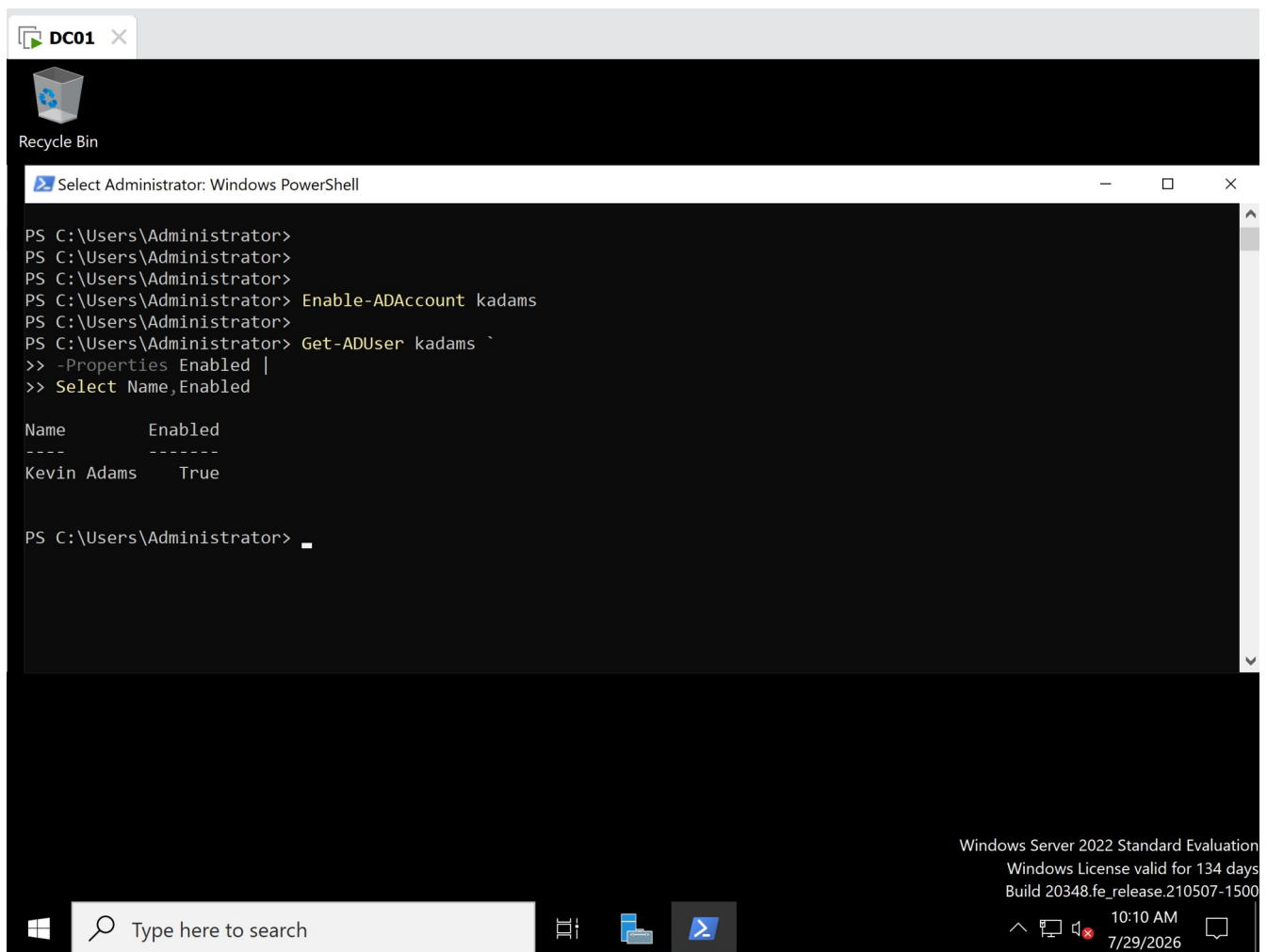
PS C:\Users\Administrator> _
```

Windows Server 2022 Standard Evaluation
Windows License valid for 134 day
Build 20348.fe_release.210507-150

Type here to search

10:06 AM
7/29/2026

Account disabled (Enabled = False)



Key Concepts Learned

Lessons Learned

This exercise demonstrated how PowerShell simplifies routine Active Directory administration. Rather than navigating through Active Directory Users and Computers, administrators can enable or disable user accounts with a single cmdlet and immediately verify the results. These commands are especially valuable for automating administrative tasks involving multiple users and are commonly used in enterprise environments.

Administrator's Notes

This task reinforced the importance of **verification after every administrative change**. Instead of assuming a command completed successfully, the account status was checked before disabling, after disabling, and again after re-enabling. This verify-first approach helps administrators ensure that changes have been applied correctly and is considered a best practice in enterprise Windows Server administration.